


Universidad Nacional de Córdoba




Facultad de Matemática, Astronomía, Física y Computación

# Bienvenido al infiernoCompletar: poner un titulo fachero

por

**Santiago Fada**

Presentado ante la **FACULTAD DE MATEMÁTICA, ASTRONOMÍA, FÍSICA Y COMPUTACIÓN** como parte de los requerimientos para la obtención del grado de Licenciado en Matemática Aplicada de la

**UNIVERSIDAD NACIONAL DE CÓRDOBA**

Octubre, 2026

Directores: Dr. Juan Bautista Cabral
Dr. Mariano Ferrero



Santiago Fada © 2026 by Santiago Fada is licensed under Creative Commons Attribution-ShareAlike 4.0 International

Certifico que el trabajo incluido en este documento es el resultado de tareas de investigación originales y que no ha sido presentado para optar a un título en ninguna otra Universidad o Institución.

Santiago Fada

# Reconocimientos

Esta Tesis fue desarrollada gracias a la ayuda económica de ...

# Agradecimientos

Completar: COMPLETAR

# Resumen

Completar: Al final, no olvidar la version en ingles

# Abstract

Completar: COMPLETING IN ENGLISH

# Índice general

**Reconocimientos** **I**

**Agradecimientos** **I**

**Resumen** **III**

**Abstract** **IV**

**1. Introducción** **1**
1.1. Contexto . . . . . 1
1.2. El problema . . . . . 2
1.3. Objetivo . . . . . 3
1.4. Resultados originales presentados . . . . . 3
1.5. Organización del trabajo . . . . . 3

**2. Antecedentes** **4**
2.1. Evolución de los centros de distribución . . . . . 4
2.2. La digitalización y el *warehouse* moderno . . . . . 8
2.3. Operaciones dentro de un *warehouse* . . . . . 9
2.3.1. Dónde se decide el costo . . . . . 10
2.3.2. La geometría del depósito . . . . . 11
2.3.3. Order picking . . . . . 12
2.3.4. Otras decisiones de diseño . . . . . 14
2.4. Perfil de actividad y demanda . . . . . 15
2.5. Asignación de ubicaciones . . . . . 18
2.5.1. Dedicado o compartido . . . . . 18
2.5.2. Las reglas de asignación . . . . . 19
2.5.3. Picks, reposiciones y rotación . . . . . 19
2.5.4. Clases y restricciones . . . . . 20
2.5.5. Lo que queda fuera de la cuenta . . . . . 21
2.6. Afinidad entre productos . . . . . 22
2.7. Formulación matemática . . . . . 24
2.8. Estado del arte . . . . . 28
2.9. Síntesis y problemas abiertos . . . . . 30

**Bibliografía** **35**

**Índice de figuras** **38**

**Índice de tablas** **40**

**Glosario** **41**

# Capítulo 1

# Introducción

Para quien compra, una operación en línea dura segundos, se elige un producto, se confirma, y al día siguiente llega a la puerta. Detrás del clic hay un depósito donde alguien camina hasta una estantería, toma el producto, lo lleva a una mesa de empaque y lo despacha, y sobre ese recorrido pesa una decisión más influyente de lo que parece, y que este trabajo busca mejorar, dónde se guarda cada producto.

## 1.1. Contexto

El comercio electrónico creció de forma sostenida durante la última década, y con él la exigencia sobre la infraestructura que lo sostiene (Boysen et al., 2019). Esa infraestructura son los centros de distribución, que organizan su operación en cinco grandes etapas encadenadas, la recepción de la mercadería que llega de los proveedores, su almacenamiento en las estanterías, el *picking* o recolección de los artículos que componen cada pedido, el empaquetado, y el despacho hacia el cliente. Ese crecimiento cambió la operación en cuatro aspectos fundamentales. Los pedidos se multiplicaron y, al mismo tiempo, se achicaron, donde antes se despachaban *pallets* completos, cientos de unidades de un mismo producto con destino a un comercio, ahora se preparan pedidos de unos pocos artículos distintos para un cliente final (Boysen et al., 2019). A eso se sumaron plazos de entrega cada vez más cortos y un catálogo que no dejó de ampliarse, de modo que el depósito pasó a resolver muchos pedidos chicos, variados y urgentes sobre un universo de productos cada vez mayor.

En la mayoría de los depósitos esa recolección sigue siendo manual, bajo un esquema conocido como *picker-to-parts* en el que es el operario quien se desplaza hasta la ubicación del producto, y no el producto el que llega hasta él (de Koster et al., 2007). Bajo ese esquema el *picking* concentra la mayor parte del costo operativo del depósito, y dentro de esa etapa la mayor parte del tiempo no se va en tomar los artículos ni en empaquetarlos, sino en caminar entre ubicaciones (de Koster et al., 2007; Tompkins et al., 2010). Esa caminata depende directamente de cómo estén distribuidos los productos en el inventario.

## 1.2. El problema

La distancia que recorre un operario para completar un pedido no depende de una sola decisión. Son cuatro las decisiones del depósito que la determinan, y se toman en momentos y niveles distintos, algunas una sola vez al diseñarlo y otras todos los días al operarlo (Gu et al., 2007; de Koster et al., 2007).

1. El *layout* fija la geometría del depósito, cuántos pasillos hay, cómo se conectan, en qué sentido se circula y dónde está la zona de despacho. Se decide al construirlo y rara vez se modifica.

2. El *batching* agrupa varios pedidos para recolectarlos en una sola pasada. Se decide todos los días, con los pedidos que van llegando.

3. El *routing* determina en qué orden se visitan las ubicaciones de un mismo recorrido. Se decide para cada recorrido, una vez armado el lote.

4. El *slotting* decide qué ubicación ocupa cada producto. Se revisa cada vez que entra inventario nuevo o cuando se reorganiza el depósito.

Este trabajo aborda principalmente la última de esas decisiones, el *slotting*, que en muchas situaciones se resuelve de manera heurística, y ante la llegada de nuevo inventario la asignación responde a la disponibilidad inmediata de espacios libres o, en el mejor de los casos, a la rotación individual de cada artículo, de modo que los productos que más se piden se ubican cerca de la zona de despacho.

La rotación es un criterio razonable pero incompleto, porque mira cada producto de forma individual y no tiene en cuenta cómo quedan ubicados unos respecto de otros. Y los productos no se demandan de forma aislada, existen patrones de compra recurrentes, o *afinidades*, en los que ciertos artículos tienden a aparecer juntos en los mismos pedidos. Dos artículos que entran en la misma promoción, por ejemplo, empiezan a aparecer en el mismo pedido de un día para el otro sin que ninguno de los dos figure entre los más vendidos del depósito, de modo que un criterio guiado solo por rotación puede dejarlos en extremos opuestos y obligar al operario a recorrer esa distancia en cada pedido que los incluya. La rotación y la afinidad son, entonces, dos señales distintas, y atender solo a la primera deja sin resolver la mitad del problema.

A esto se suma que las afinidades no son estables. Los hábitos de consumo cambian con el tiempo, lo que vuelve obsoleta cualquier organización fijada de una vez y para siempre. Cuando la promoción del ejemplo anterior termina, el par se disuelve aunque los dos productos sigan en el catálogo, y la ubicación que compartían deja de tener sentido.

Frente a ese panorama hay un dato alentador, y es que las afinidades no hay que adivinarlas. Los sistemas de gestión de almacenes guardan el historial completo de órdenes, y en ese registro está la estructura de la demanda, qué se pide, con qué frecuencia, y junto con qué.

Ahora bien, sacarle provecho a ese registro está lejos de ser inmediato. Se trata de elegir, entre todas las formas posibles de asignar productos a ubicaciones, aquella que minimiza la distancia esperada de recolección, y ahí es donde aparece un problema de optimización combinatoria. Con $n$ productos para repartir en $n$ ubicaciones hay $n!$ asignaciones posibles, así que a la escala de un depósito recorrerlas todas es inviable. Ni siquiera los métodos exactos que evitan

recorrerlas una por una llegan mucho más allá de unas decenas de productos (Loiola et al., 2007), muy lejos de las decenas de miles de artículos que hay que ubicar en un depósito real.

## 1.3. Objetivo

El objetivo de este trabajo es decidir la ubicación de cada producto combinando dos señales que se estiman del historial de pedidos, cuánto se pide cada artículo y con qué intensidad se piden juntos dos de ellos. En la literatura el problema se conoce como *storage location assignment problem* (SLAP), y ponderar ambas señales en una única función de costo lo ubica dentro de la familia del *quadratic assignment problem* (QAP) (Rojas Reyes et al., 2019; Koopmans and Beckmann, 1957). Esa familia es NP-hard (Loiola et al., 2007), de modo que a la escala de un depósito real, con decenas de miles de productos, resolverla de forma exacta queda fuera de alcance.

El trabajo no busca entonces la solución óptima, sino optimizar un problema similar y tratable que la aproxime. La validez de cada estrategia se mide simulando los recorridos sobre un período posterior al utilizado para estimar las señales, y comparando la distancia resultante con la que produce la asignación vigente del depósito. Aunque la evaluación se realiza sobre un depósito concreto, el método no depende de su geometría ni de su catálogo, sino únicamente de contar con un historial de pedidos y un mapa de distancias, de modo que el mismo procedimiento puede aplicarse a cualquier operación que registre esos datos.

## 1.4. Resultados originales presentados

Completar: SI HAY RESULTADOS COMO CONFERENCIAS O PAPERS, TAMBIEN PUEDO PONER SI SE IMPLEMENTO EN LA EMPRESA O SIMILAR

## 1.5. Organización del trabajo

Los capítulos de este trabajo se organizan de la siguiente manera.

El Capítulo 2 reúne el contexto necesario para ubicar el problema, los fundamentos y las técnicas sobre las que se apoya el trabajo, junto con la revisión de la literatura relevante.

Completar: ORGANIZACION: describir los capitulos restantes (deposito, optimizacion, resultados y conclusiones) cuando su alcance este definido

# Capítulo 2

# Antecedentes

Este capítulo reúne los antecedentes necesarios para situar el problema abordado en este trabajo, desde la evolución de los sistemas de almacenamiento hasta las técnicas actuales de asignación de ubicaciones.

Para entender cómo se llega a este problema, conviene empezar por el depósito[1], que tal como se lo conoce hoy, despachando miles de pedidos por día y registrando sus movimientos, no apareció de un día para el otro, sino que es el resultado de décadas de cambios en lo que se espera de un almacén, y recorrer esa evolución permite entender por qué hoy el *picking*, la tarea de recolectar los productos que componen cada pedido, concentra buena parte del esfuerzo operativo.

Dentro de esa tarea, además, una parte importante del tiempo se dedica a caminar entre ubicaciones. De ahí surge una pregunta bastante natural, ¿qué se puede hacer para que los operarios caminen menos? Hay distintas respuestas, y dos de ellas dependen directamente de cómo se distribuyen los productos, ubicar en los lugares más convenientes aquellos que más se piden y mantener cerca entre sí los que suelen aparecer juntos en un mismo pedido.

Ambas ideas son fáciles de enunciar, aunque no igual de fáciles de resolver. Bajo ciertos supuestos, la primera conduce a un problema de asignación relativamente sencillo, mientras que la segunda requiere considerar las relaciones entre productos y lleva a un problema de optimización mucho más difícil de resolver a escala real. Esa diferencia ayuda a entender los distintos caminos que siguió la literatura y las preguntas que todavía quedan abiertas.

## 2.1. Evolución de los centros de distribución

Durante buena parte de su historia, un depósito fue principalmente un lugar donde la mercadería esperaba. Los productos llegaban, se almacenaban y permanecían allí hasta que fueran necesarios. Esto permitía producir en un momento y vender en otro, algo especialmente importante cuando la demanda se concentraba en ciertas épocas del año. Un fabricante de juguetes, por ejemplo, podía producir durante varios meses y acumular stock para responder al aumento de ventas durante Navidad. Al mismo tiempo, mantener inventario

[1] En este trabajo, los términos depósito, almacén, centro de distribución y *warehouse* se utilizan de manera intercambiable, salvo cuando sea necesario distinguirlos.

daba cierto margen frente a cambios inesperados en la demanda (Bartholdi and Hackman, 2014, sección 1.1).

En ese contexto, la principal preocupación era encontrar espacio para guardar la mercadería y aprovecharlo bien. Como los productos se movían con menos frecuencia, la ubicación exacta de cada uno dentro del depósito tenía una importancia mucho menor que la que tiene en una operación moderna (Ballou, 2007).

Ese equilibrio empezó a cambiar a medida que crecieron los volúmenes de producción y de mercadería almacenada (Ballou, 2007). Ya no alcanzaba con disponer de espacio suficiente, también era necesario mover los productos dentro del depósito de forma cada vez más eficiente. Así, el problema comenzó a desplazarse desde dónde guardar la mercadería hacia cómo manipularla y transportarla.

Una de las respuestas llegó por el lado de la estandarización, con la difusión de la carga unitaria a mediados del siglo XX. Un *pallet* es una plataforma de medidas normalizadas sobre la que se apila mercadería para manipularla como un bloque único, y su adopción masiva, junto con la del montacargas, convirtió el movimiento de mercadería en una operación mecanizable y previsible (Bartholdi and Hackman, 2014, sección 5.1). Esta forma de manipular la mercadería también consolidó durante décadas una operación pensada alrededor de grandes unidades de carga, algo que cambiaría profundamente con la expansión del comercio electrónico.

Con la carga estandarizada, mover mercadería entre fábricas, depósitos y comercios dejó de ser un problema de cada uno y pasó a poder planificarse en conjunto. Sobre esa base se constituyó la logística como disciplina durante la segunda mitad del siglo, entendida como el estudio de cómo organizar esa cadena de principio a fin. Lo que hasta entonces eran actividades sueltas, almacenar, transportar, reponer, empezó a tratarse como un sistema con objetivos propios y costos medibles (Ballou, 2007).

Pensar la cadena entera deja ver además otro motivo para tener depósitos, menos evidente que el de guardar mercadería y de naturaleza casi aritmética, que es la consolidación del transporte (Bartholdi and Hackman, 2014, sección 1.1). Si $m$ proveedores deben abastecer directamente a $n$ comercios hacen falta del orden de $m \times n$ envíos, cada uno relativamente chico y por lo tanto caro por unidad transportada, mientras que si toda la mercadería pasa por un depósito intermedio alcanzan $m + n$ envíos, cada uno más grande y con el camión más lleno. Con cien proveedores y cien comercios la diferencia es entre diez mil conexiones posibles y apenas doscientas a través de un punto intermedio. Concentrar la mercadería en un punto tiene además otras ventajas, como aprovechar descuentos por compra en volumen o postergar la configuración final de un producto genérico hasta saber qué mercado lo pide.

A partir de los años setenta vino otro giro, conceptual antes que tecnológico, con la difusión de las filosofías de producción ajustada, nacidas en la industria automotriz japonesa y basadas en producir solo lo que se va a vender y cuando se va a vender (Womack et al., 1990). Mantener grandes cantidades de inventario también tiene un costo, el mismo ocupa espacio, requiere recursos para su almacenamiento y queda expuesto a roturas, deterioro u obsolescencia. Por eso, el almacenamiento comenzó a verse no solo como un respaldo frente a la incertidumbre, sino también como capital inmovilizado y como una fuente de costos.

Las consecuencias para lo que aquí importa son dos. Por un lado el depósito dejó de ser un lugar donde la mercadería descansa y pasó a ser uno donde circula, lo que vuelve al movimiento interno un costo mucho más importante en lugar de una externalidad menor. Por otro lado, al aumentar la frecuencia de los movimientos, comenzó a cobrar importancia una característica hasta entonces menos relevante para organizar el espacio, qué productos se solicitaban con mayor frecuencia. Esa información podía utilizarse para ubicar los artículos de mayor rotación en posiciones más convenientes.

En ese contexto se consolidó la figura del centro de distribución, que se distingue del almacén tradicional menos por su tamaño que por aquello que mide. Mientras que en el almacén tradicional la capacidad de almacenamiento ocupaba un lugar central, en un centro de distribución adquieren mayor importancia la cantidad de pedidos procesados, el tiempo necesario para hacerlo y el costo asociado a cada uno.

Hablar de centros de distribución, sin embargo, todavía abarca operaciones que se parecen poco entre sí, y la distinción más útil pasa por el tipo de cliente al que sirven (Bartholdi and Hackman, 2014, sección 1.2). Uno que abastece tiendas trabaja contra un cliente más regular y previsible, con órdenes que pueden tener cientos o miles de artículos y que suelen conocerse con un día o más de anticipación, de modo que hay margen para planificar. Un depósito de repuestos enfrenta el problema opuesto, porque llega a almacenar decenas o cientos de miles de piezas cuya demanda individual es chica y difícil de anticipar, aunque la actividad en promedio resulte estable. En el extremo opuesto está el centro de distribución orientado al comercio electrónico masivo, donde las órdenes suelen ser pequeñas, en muchas ocasiones de apenas uno a tres artículos, pero llegan con una frecuencia muy alta y deben procesarse rápidamente. A eso se agregan operaciones con restricciones propias, como los depósitos de perecederos, en los que la mercadería permanece a veces solo unas horas y hay que sostener la cadena de frío y reglas de salida por vencimiento, o los operadores logísticos externos, que conviven en una misma instalación con inventario de varios clientes (Bartholdi and Hackman, 2014, sección 1.2). La Tabla 2.1 resume estas diferencias, que reaparecen cada vez que una decisión dependa del tipo de operación.

Esas diferencias no son solo de escala, porque también cambia la unidad física con la que se manipula la mercadería. Cerca de la fábrica el producto tiende a moverse en *pallets*, y a medida que avanza por la cadena de distribución esas grandes unidades se van dividiendo en cajas, paquetes internos y, finalmente, unidades individuales, que es la escala más cercana al consumidor final (Bartholdi and Hackman, 2014, sección 2.2).

Esa escala física es distinta de la identidad del producto. Cada producto o variante que el inventario administra por separado se identifica con una *stock keeping unit* (SKU), y una misma SKU puede estar a la vez en *pallets*, en cajas y en piezas sueltas, así como dos talles de un mismo artículo son dos SKU distintas. De aquí en adelante SKU se usa en ese sentido, la referencia con la que el depósito cuenta, y no la pieza que manipula.

Ese descenso es el que vuelve relevante todo lo que sigue. Mover un *pallet* entero de una ubicación a otra es un problema distinto de recorrer el depósito para juntar una o dos unidades de varios productos diferentes, y cuanto más chica es la unidad que se manipula y más pedidos hay que preparar, más peso adquiere la decisión de dónde ubicar cada artículo.

| **Tipo** | **Cliente** | **Pedido típico** | **Surtido y demanda** | **Unidad que sale** | **Lo que manda** |
|---|---|---|---|---|---|
| Reposición de tiendas | tiendas de una cadena, fijas y con entregas programadas | cientos o miles de líneas, conocidas con un día o más de anticipación | cambia con el marketing, demanda agregada alta y planificable | caja o *pallet* | mover mucho volumen según un plan |
| Repuestos | concesionarios y talleres | dos corrientes, reposiciones grandes y previsibles, y urgencias de dos o tres líneas | decenas o cientos de miles de piezas, demanda individual chica e impredecible, mucho stock de seguridad | pieza | variedad, urgencia y espacio |
| Comercio electrónico | consumidor final | una a tres líneas, muchísimos pedidos por día, despacho inmediato | surtido amplio, demanda que se intenta moldear con precios y plazos, devoluciones | pieza | muchos pedidos chicos con plazo corto |
| Perecederos | comercios y consumidores | variable | permanencia de horas, salida por vencimiento, zonas de temperatura distintas | caja | frío, espacio caro y rotación forzada |
| Operador logístico | varios clientes en un mismo edificio | la de cada cliente | inventarios separados, estacionalidades que se complementan | mixta | escala compartida y separación de dueños |

Tabla 2.1: Tipos de centro de distribución según el cliente al que sirven, y lo que cada uno implica para el pedido, el surtido y la unidad que se despacha. Elaborado a partir de Bartholdi and Hackman (2014, sección 1.2) y, para las devoluciones del comercio electrónico, de Boysen et al. (2019).

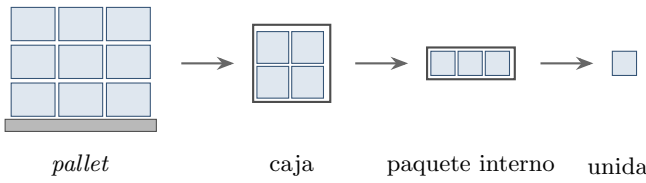



Figura 2.1: La unidad de manipulación se reduce a medida que el producto avanza por la cadena de distribución.

## 2.2. La digitalización y el *warehouse* moderno

El aumento en la cantidad de productos, ubicaciones y movimientos hizo que muchos aspectos que antes parecían irrelevantes fueran cada vez más importantes, y al mismo tiempo volvió mucho más difícil administrar el depósito de forma manual. Un centro de distribución grande puede ocupar decenas de miles de metros cuadrados, almacenar alrededor de cien mil productos distintos y emplear a cientos de personas que preparan miles de pedidos por día (Bartholdi and Hackman, 2014, capítulo 4).

A esa escala ya no alcanza con conocer el depósito por experiencia o recordar dónde está cada producto, porque hace falta un registro actualizado del inventario, es decir de qué mercadería hay, en qué cantidad, de dónde está guardada y de los movimientos que ocurren todo el tiempo dentro de la instalación. Ese registro lo lleva hoy el sistema de gestión de almacenes, el *warehouse management system* (WMS), aunque su capacidad más básica no es esa sino una contable, registrar la entrada y la salida de mercadería, porque la recepción de productos dispara el pago a los proveedores y la salida dispara la facturación al cliente (Bartholdi and Hackman, 2014, sección 4.1). Sobre esa base se fue construyendo todo lo demás.

El siguiente salto de funcionalidad consiste en registrar no solo cuánto queda de cada producto sino también en qué ubicación está. Eso es lo que la literatura llama *stock locator system*, y es lo que le permite al software dejar de limitarse a transacciones contables para dirigir la operación diaria (Bartholdi and Hackman, 2014, sección 4.2). Con esa información el sistema puede dirigir tanto el *put-away*, que es guardar en una ubicación lo que acaba de recibirse, indicando dónde va cada SKU, como el *picking*, indicando de dónde retirarla, y sobre esa misma representación del almacén se apoyan funciones cada vez más amplias, desde coordinar reposiciones hasta distribuir trabajo entre los operarios.

Con esas capacidades, procesar un pedido ya no exige recorrer sus productos en el orden en que el cliente los pidió. El propio sistema recibe las órdenes y las transforma en una lista de recolección, que es la lista de ubicaciones que el operario tiene que visitar, ordenada según el recorrido por el depósito y no según el pedido, de modo que es esa lista, y no la memoria del operario, la que le dice adónde ir y en qué orden.

Para que el inventario de ubicaciones sea confiable, cada movimiento, una recepción, un guardado, una extracción, tiene que quedar registrado en el momento en que ocurre, y eso es lo que hizo posible el código de barras. Con su difusión a partir de los años setenta, identificar un producto o una ubicación pasó a ser escanear una etiqueta con una terminal en vez de transcribir un número a mano (Brown, 1997), y como cada escaneo ocurre en el momento y

en el lugar del movimiento que registra, el inventario del sistema pudo acercarse mucho más al estado real del depósito. Ese acercamiento no lo logra el código de barras por sí solo, porque hace falta además que las terminales de escaneo sean portátiles y acompañen al operario hasta donde ocurre el movimiento, que la base de datos absorba actualizaciones simultáneas desde recepción, *picking* y despacho, y que los procedimientos obliguen a escanear en cada paso (Bartholdi and Hackman, 2014, sección 4.2). Cuando todo eso está, el nivel de detalle es fino, al punto de que un buen WMS rastrea cada lugar donde puede haber producto, incluidas las horquillas de cada montacargas.

El alcance de estos sistemas sigue creciendo, y el WMS se relaciona con los sistemas del resto de la empresa, de los que recibe las órdenes y a los que devuelve un resumen de lo que ocurre puertas adentro, integrado en la planificación de la cadena de suministro completa (Bartholdi and Hackman, 2014, capítulo 4). Ese control es una de las razones por las que la cadena de suministro se aceleró tanto entre los años noventa y la década de 2010, porque, como argumentan Bartholdi and Hackman (2014, capítulo 4), un producto controlado con precisión se mueve más rápido y con menos inventario en el sistema, una mejora que se suma a las del transporte y de la coordinación entre eslabones.

Todo ese registro tiene además un efecto secundario, y es que al anotar cada movimiento el depósito acumula sin proponérselo el historial completo de qué se pidió, cuándo, en qué cantidad y junto con qué otra cosa, un historial que es la materia prima de buena parte de lo que sigue.

Sobre esa base informatizada aterrizó el comercio electrónico, que al cambio de escala le sumó complicaciones que hasta entonces eran marginales, ya que una parte de lo despachado empezó a volver en forma de devoluciones y una misma instalación pasó a atender varios canales a la vez, reponiendo tiendas y despachando a clientes finales desde el mismo edificio, aunque los dos tipos de pedido se parezcan poco (Boysen et al., 2019). Las devoluciones en particular invirtieron el sentido del flujo, porque cada paquete que vuelve tiene que ser recibido, controlado y guardado otra vez con el mismo cuidado que un producto recién llegado del proveedor, con la diferencia de que llega de a una unidad, en un estado que nadie conoce de antemano y sin que ningún plan lo haya previsto.

La industria ensayó dos respuestas a esa presión, y la primera fue automatizar el movimiento interno para que el producto viaje hasta un operario que trabaja quieto, el esquema *parts-to-picker*, con carruseles que giran, grúas que recorren pasillos de almacenamiento denso y, más recientemente, robots móviles que cargan estanterías enteras hasta una estación de *picking* (Boysen et al., 2019). La otra fue mantener al operario caminando hasta el producto, el esquema *picker-to-parts* de la introducción, y repensar cómo se organiza su trabajo y el espacio, porque exige poca inversión, se adapta a cualquier surtido y escala agregando personas. Este trabajo estudia un depósito del segundo tipo, y todo lo que sigue se refiere a ese esquema.

## 2.3. Operaciones dentro de un *warehouse*

Para optimizar cualquiera de las decisiones que se toman puertas adentro hace falta entender primero las operaciones del depósito, y la manera más natural de hacerlo es seguir el camino que recorre la mercadería desde que baja de un camión hasta que sale despachada hacia un cliente. Esta sección recorre

ese camino preguntándose en cada etapa qué cuesta y dónde se decide ese costo, junto con el edificio y el equipamiento donde todo ocurre, hasta desembocar en la etapa que se lleva la mayor parte, el *order picking* o simplemente *picking*, que consiste en recorrer el depósito para reunir los productos que componen cada pedido y llevarlos de vuelta hacia el *dock*, las puertas por donde salen los camiones, para empacarlos y despacharlos, para cerrar con las otras decisiones de diseño con las que convive la que estudia este trabajo.

### 2.3.1. Dónde se decide el costo

El camino de la mercadería tiene seis etapas, que la Figura 2.2 sigue sobre la planta de un depósito, y empieza con la recepción (1), que es descargar la mercadería del camión y controlar que sea lo que se esperaba, seguida del *put-away* (2), que es llevar cada producto a una ubicación y guardarlo ahí. Después viene el almacenamiento (3), que no es una tarea sino el tiempo que la mercadería espera en esa ubicación hasta que algún pedido la requiere, y recién entonces el *picking* (4), que recolecta de esas ubicaciones los productos de cada pedido. Lo recolectado pasa por el empaque (5), donde se controla y se arma el bulto que va a viajar, y termina en el despacho (6), donde los bultos se cargan en el camión de salida.

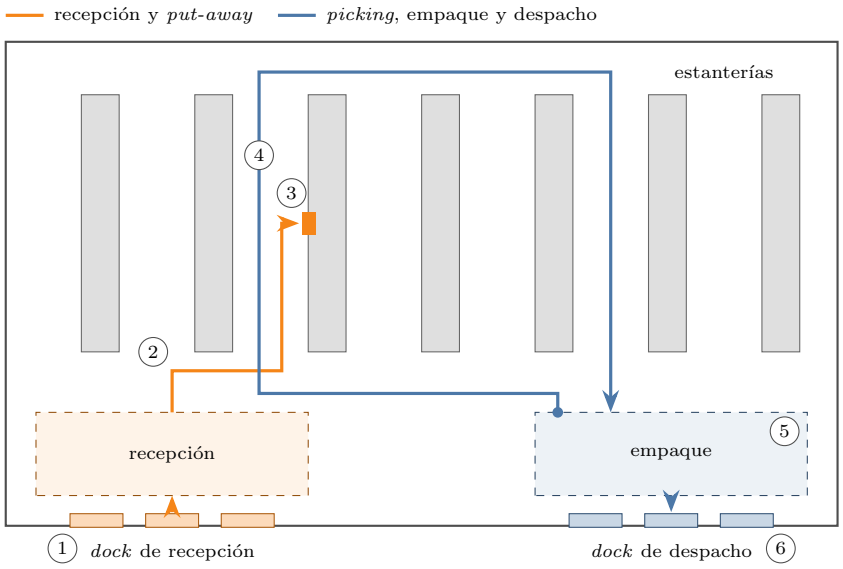



Figura 2.2: Planta esquemática de un depósito y el camino de un producto. Entra por el *dock* de recepción (1), el *put-away* lo lleva hasta una ubicación de las estanterías (2), donde queda almacenado (3) hasta que un recorrido de *picking*, que sale de empaque (punto azul), pasa a buscarlo (4) y lo deja en empaque (5), de donde sale por el *dock* de despacho (6). En naranja el tramo de entrada, en azul el de salida.

El recorrido empieza y termina en el *dock*, que es el sector de puertas del depósito donde se descargan los camiones que llegan y, más tarde, se cargan los que salen, de modo que todo lo que entra y sale pasa por ahí (Bartholdi and

Hackman, 2014, capítulo 3). La mercadería que se descarga llega todavía en las unidades grandes de la Sección 2.1, *pallets* o cajas cerradas, y por eso recibirla exige comparativamente poco trabajo por unidad.

Una parte importante del costo posterior se condiciona un paso más adelante, en el *put-away*. Antes de guardar hay que elegir dónde, y de esa elección depende en buena medida cuánto habrá que caminar después para volver a buscar cada producto, de modo que una etapa fija un costo que otra paga. Por otro lado, a la espera de que algún pedido solicite una SKU, la mercadería no consume trabajo pero sí espacio, ocupando una ubicación con dirección propia dentro del depósito.

Del lado de la salida, lo recolectado pasa por el control y el empaque antes de volver al *dock*, y ahí detectar un error todavía es barato, porque un pedido mal despachado cuesta el envío de vuelta, el reproceso y un cliente decepcionado.

Una regla atraviesa todas las etapas, y es que el producto fluya deteniéndose lo menos posible, porque cada vez que un bulto se deja en algún lugar intermedio alguien va a tener que volver a levantarlo, y esa doble manipulación, sumada sobre los cientos de miles de piezas que pasan por un depósito, se vuelve un costo considerable (Bartholdi and Hackman, 2014, sección 2.1).

La Tabla 2.2 le pone números a ese reparto. Son las estimaciones de Tompkins et al. (2010), un manual clásico de planificación de instalaciones que la literatura posterior sigue usando (de Koster et al., 2007), y conviene leerlas como órdenes de magnitud antes que como medidas, porque la proporción exacta depende de qué se almacena y a quién se atiende, mientras que el orden entre las etapas se mantiene en casi todos los casos reportados.

| **Etapa** | **Participación en el costo operativo** |
|---|---:|
| Recepción | 10 % |
| Put-away | 15 % |
| Order picking | 55 % |
| Control, empaque y despacho | 20 % |

Tabla 2.2: Reparto típico del costo operativo entre etapas, según las estimaciones clásicas de Tompkins et al. (2010) reproducidas en Bartholdi and Hackman (2014, capítulo 3).

### 2.3.2. La geometría del depósito

La geometría de un depósito, lo que la literatura llama *layout*, es cómo están dispuestos sobre la planta los pasillos, las estanterías y los *docks*, y es de las decisiones más difíciles de revertir, porque se fija al construir el edificio y sus consecuencias permanecen durante décadas. El esquema dominante es sencillo, con la superficie dividida en pasillos paralelos flanqueados por estanterías más el *dock*, y la Figura 2.3 muestra esa forma básica junto con sus variantes principales. Sobre esa base, cada elección de diseño es en el fondo un problema de optimización cuyas consecuencias se pagan o se cobran en cada recorrido posterior.

Una de esas elecciones es cuántos *docks* tener y dónde ponerlos, y su efecto es menos obvio de lo que parece, porque no cambia cuánto espacio hay sino cómo se reparte la conveniencia entre las ubicaciones (Bartholdi and Hackman, 2014,

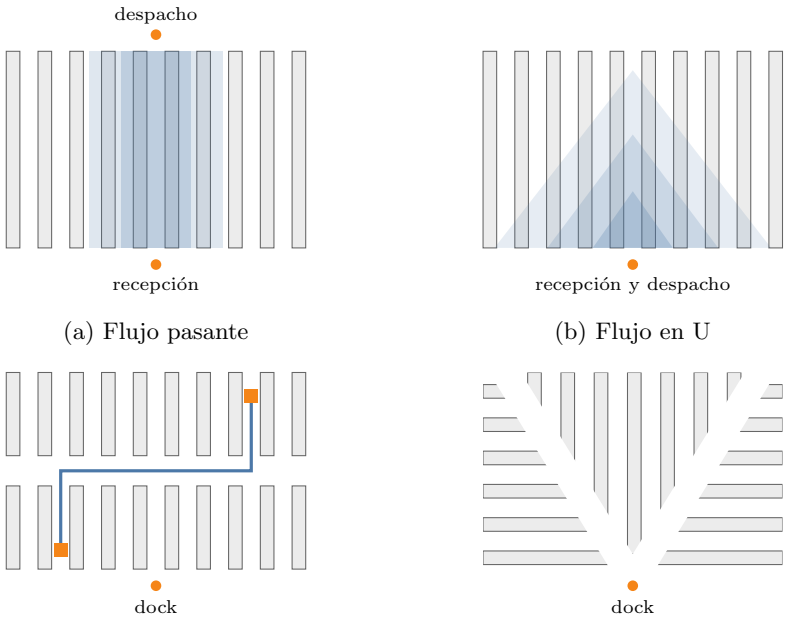
sección 6.2). Cuando la recepción y el despacho comparten un mismo *dock*, en la configuración conocida como flujo en U de la Figura 2.3b, unas pocas ubicaciones cercanas quedan muy convenientes y las esquinas opuestas quedan muy castigadas, mientras que si hay un *dock* de entrada y otro de salida en lados opuestos, en el flujo pasante de la Figura 2.3a, el producto cruza el edificio y muchas ubicaciones resultan igual de razonables, aunque ninguna excepcional. Ninguna de las dos configuraciones domina a la otra, y el criterio es elegir según el sesgo de la demanda, porque una operación donde pocas SKU concentran la actividad aprovecha mejor el puñado de posiciones excelentes del flujo en U, mientras que una de actividad más repartida prefiere la uniformidad del flujo pasante. El patrón del argumento es lo más valioso que deja esta elección, la geometría define qué ubicaciones valen más, y la demanda define cuánto conviene esa definición.

Los pasillos admiten sus propias variantes. En general conviene orientarlos en la dirección del flujo, y es habitual agregar pasillos transversales que permiten cortar camino entre ubicaciones, como en la Figura 2.3c, al precio de resignar posiciones de almacenamiento y de alargar levemente los viajes que los cruzan. Se ha propuesto incluso abandonar el paralelismo, porque reorientar algunos pasillos en diagonal hacia el *dock*, en la espina de pescado de la Figura 2.3d, acorta el viaje entre un punto central de despacho y cada ubicación, con reducciones de hasta un 20 % en depósitos de carga unitaria, donde cada viaje es de ida y vuelta a una sola ubicación, a cambio de un edificio algo más grande (Gue and Meller, 2009). Esa ventaja se achica cuando un mismo recorrido visita muchas ubicaciones, pero que un cambio de ángulo pueda mover tanto la cuenta da una idea de cuánto pesan decisiones que parecen puramente arquitectónicas.

El equipamiento que llena esa geometría responde siempre a alguno de dos propósitos, ahorrar trabajo o aprovechar espacio (Bartholdi and Hackman, 2014, capítulo 5), y cuál de los dos pesa más depende de la unidad que se manipula. Mientras se mueven *pallets* el equipo se elige por densidad, y cuánto guardar por metro cuadrado es en sí mismo una decisión de optimización que se retoma en la Sección 2.3.4. Cuando la unidad baja a la caja o a la pieza suelta, que es el caso del picking de pedidos chicos, el equipo se elige en cambio por trabajo, y lo que se busca es que la mayor cantidad posible de productos quede al alcance de la mano en el frente del pasillo, porque cuanto más se recolecta por metro caminado menos cuesta cada pedido (Bartholdi and Hackman, 2014, capítulo 5).

### 2.3.3. Order picking

El *picking* se lleva la mayor parte del costo operativo porque es la etapa donde la mercadería se fragmenta, ya que un *pallet* que se recibió y se guardó una sola vez se va después de a cajas o de a piezas repartidas en muchos pedidos, de modo que una sola entrada genera múltiples extracciones, aunque una misma visita pueda llevarse varias unidades o servir a más de un pedido, y cuanto más chica es la unidad que se extrae más manipulación exige y más difícil resulta automatizarla (de Koster et al., 2007). Dentro de esa etapa, además, las estimaciones de Tompkins et al. (2010) ubican alrededor de la mitad del tiempo del operario en caminar de una ubicación a la siguiente en lugar de extraer, un viaje que no le agrega valor a ningún pedido y al que apunta buena parte del diseño de la operación.




(a) Flujo pasante

(b) Flujo en U

(c) Pasillo transversal

(d) Espina de pescado

Figura 2.3: Variantes de geometría de un depósito. La ubicación de los *docks* y la orientación de los pasillos cambian qué ubicaciones resultan convenientes, y en (a) y (b) el sombreado más oscuro indica mayor conveniencia. Esquemas adaptados de Bartholdi and Hackman (2014, capítulo 6).

Como el viaje es el costo más importante, la cantidad de líneas de una lista de recolección es una buena medida de su trabajo, pero no la única, porque una lista cuyas líneas caen cerca unas de otras se recolecta barata mientras que una de pocas líneas dispersas paga mucho viaje por cada extracción, una propiedad que se conoce como densidad de picking.

Sobre el esquema *picker-to-parts* operan, además de dónde está cada producto, las decisiones que siguen.

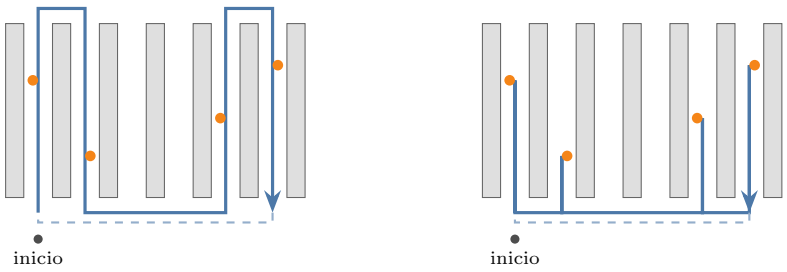
La primera es cuántos pedidos recolecta un operario por pasada, lo que se conoce como *batching*. Agrupar varios reparte la caminata entre más líneas, pero introduce el costo de separar qué artículo pertenece a qué pedido, ya sea clasificando mientras se recolecta o en una estación posterior (Bartholdi and Hackman, 2014, sección 3.3). La economía del asunto depende del tamaño del pedido, dado que los de una sola línea son los más fáciles de agrupar, porque cada extracción es ya un pedido completo y no hay nada que reordenar, aunque igual haya que identificar cada unidad con su orden, darle un lugar en el carro y llevarla a empacar, mientras que los muy grandes se bastan solos porque ya tienen densidad de picking propia, y el problema real son los intermedios, con demasiadas líneas para ignorar el viaje y demasiado pocas para amortizarlo. También puede repartirse un mismo pedido entre varios operarios que trabajan en zonas distintas del depósito, lo que acorta los recorridos y el tiempo de flujo de cada orden, al precio de tener que consolidar después las partes.

La segunda decisión es en qué orden visitar las ubicaciones de una lista. Formalmente es un problema de viajante de comercio (*travelling salesman problem*, TSP), el de encontrar, dado un conjunto de puntos y las distancias entre ellos, el recorrido más corto que los visite a todos y vuelva al inicio. Enunciado así parece inocente, pero la cantidad de recorridos posibles crece de forma factorial, con $n$ puntos hay $(n-1)!/2$, de modo que una lista de diez ubicaciones admite unos ciento ochenta mil recorridos y una de veinte más de sesenta mil billones, y probarlos todos deja de ser una opción mucho antes de llegar al tamaño de una lista real. Tampoco se conoce un atajo general, porque no hay ningún algoritmo conocido que encuentre el mejor recorrido en un tiempo que crezca como un polinomio en $n$, y hay buenas razones para creer que no existe (Karp, 1972). Eso no impide resolverlo en la práctica, porque los métodos exactos modernos combinan cotas con una enumeración inteligente que descarta familias enteras de recorridos sin mirarlos, y llegan a instancias de decenas de miles de puntos (Applegate et al., 2006), pero sí impide que un operario con una lista en la mano lo resuelva a ojo, y es ahí donde el depósito ayuda. Los puntos de una lista no están en cualquier lado sino sobre pasillos paralelos, y esa estructura es la que aprovecharon Ratliff and Rosenthal (1983) para dar un algoritmo que, en un depósito rectangular de un solo bloque, encuentra el recorrido óptimo en un tiempo proporcional a la cantidad de pasillos, aunque la garantía se pierde a medida que el *layout* se complica con pasillos transversales. En la práctica, además, Bartholdi and Hackman (2014, capítulo 10) observan que ningún WMS que conozcan maneja un modelo geométrico del depósito, y que un operario necesita reglas simples que pueda seguir sin mapa. Lo habitual es entonces fijar un recorrido patrón y visitar las ubicaciones de cada lista en el orden en que ese patrón las encuentra. La más estudiada es la política en forma de $S$ (*S-shape*) de la Figura 2.4a, en la que el operario entra a cada pasillo que tenga al menos una extracción, lo recorre completo y sale por el otro extremo, salteando los pasillos vacíos (de Koster et al., 2007), con lo que un pasillo con una sola extracción cuesta lo mismo que uno lleno. Su alternativa usual es la política de retorno de la Figura 2.4b, en la que se entra y se sale de cada pasillo por el mismo corredor frontal, lo que evita recorrer la parte del pasillo que queda más allá de la última extracción a cambio de desandar cada entrada. Cuál de las dos conviene depende de cuántas ubicaciones visita una lista típica y de dónde están los productos que más se piden, que conviene tener cerca del recorrido patrón para alcanzarlos casi sin desvío.

El tamaño de la orden, que ya decidía si convenía agrupar, gobierna también cuánto vale afinar el recorrido (Bartholdi and Hackman, 2014, sección 10.3). Con listas de una a tres líneas la optimización aporta poco, porque cualquier operario encuentra solo el camino corto, y con listas que visitan muchísimas ubicaciones tampoco, porque hasta la ruta óptima termina atravesando casi todo el depósito. Esa observación excede al ruteo, porque el mismo mecanismo acota a cualquier estrategia que intente ahorrar caminata. Y hay una constante que ninguna de estas decisiones puede cambiar, dado que todas toman como un dato dónde está cada producto, y decidir eso es una palanca en sí misma, la de la Sección 2.5.

### 2.3.4. Otras decisiones de diseño

Dónde está cada producto es una de las palancas del depósito, pero no la única, y conviene ver con qué otras convive antes de detenerse en ella, porque




(a) Política en forma de $S$ (b) Política de retorno

Figura 2.4: Dos políticas de ruteo sobre el mismo layout, con las estanterías en gris y las mismas cuatro extracciones (puntos). La política en $S$ recorre completos solo los pasillos con extracciones, mientras que la de retorno entra y sale de cada uno por el corredor frontal. La línea punteada es la vuelta al inicio.

las demás fijan el terreno sobre el que esa decisión se toma. La más cercana es el área de picking rápido, que consiste en separar el almacenamiento del picking y armar un depósito chico dentro del depósito, donde se guardan en poca cantidad las SKU más pedidas para que la mayor parte de las extracciones ocurra en un espacio reducido, con menos caminata y más control, mientras el resto del stock queda en una zona de reserva desde la que se repone (Bartholdi and Hackman, 2014, capítulo 8). El costo de esa comodidad es la reposición, porque cada vez que el área rápida se queda sin una SKU alguien tiene que ir a buscarla a la reserva, y por eso las dos preguntas de diseño son qué SKU van adelante y cuánto de cada una, ya que si se les da poco espacio el ahorro en *picks* se lo comen las reposiciones. Ni darles a todas el mismo espacio ni darles a todas los mismos días de venta es lo mejor, y el reparto óptimo favorece a las más pedidas pero menos que en proporción a sus *picks* (Bartholdi and Hackman, 2014, sección 8.3).

Hay otras decisiones del mismo tipo, y alcanza con nombrarlas para ver que el *slotting* es una entre varias. Cuando se guardan *pallets* hay que elegir cuántos poner uno detrás de otro en cada fila, porque las filas largas ahorran pasillo pero dejan huecos a medida que se saca una SKU, y al despachar hay que decidir cómo armar cada bulto de salida. También hay que repartir el trabajo entre los operarios para que ninguno espere al otro, y hasta decidir si conviene almacenar, porque un *crossdock* recibe lo que ya está pedido y lo pasa de un camión a otro sin guardarlo, con lo que se ahorra el *picking* entero a cambio de mucha coordinación (Bartholdi and Hackman, 2014, capítulos 6, 9, 11 y 13). Todas estas decisiones se parecen en dos cosas, cada una se puede escribir como un problema de optimización, con un costo que se mide y una decisión que lo cambia, y todas se toman mirando el mismo historial de pedidos, que es lo que hay que saber leer antes de decidir cualquiera.

## 2.4. Perfil de actividad y demanda

Antes de decidir dónde va cada SKU hay que leer el historial que el WMS acumula, y a eso la literatura lo llama perfil de actividad (Bartholdi and Hackman, 2014, capítulo 14). Se arma con tres fuentes que salen de distintos sistemas

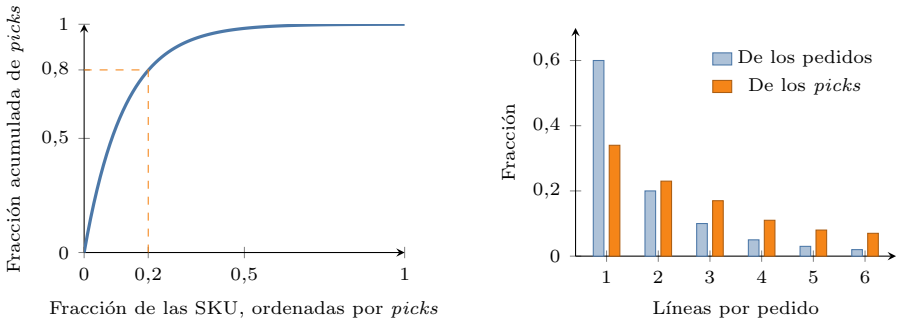
de la empresa, del catálogo salen los datos de cada SKU, qué es, cuánto mide y pesa, a qué familia pertenece y desde cuándo se vende, del sistema de ventas o del propio WMS sale el historial de pedidos, qué SKU, en qué cantidad, cuándo y en qué pedido, y del WMS sale el mapa de ubicaciones, dónde está hoy cada SKU. La advertencia es que el historial muchas veces viene del sistema financiero y no del operativo, con lo que una fecha puede ser la de la venta y no la del despacho, y una cantidad negativa suele ser una devolución.

De cada SKU se miran dos medidas distintas, en cuántos pedidos apareció y cuántas unidades se movieron. Para la primera conviene fijar qué es un *pick*. Cada vez que un operario va hasta una ubicación y extrae una SKU para una lista de recolección, eso es un pick, así que la cantidad de *picks* de una SKU es la cantidad de listas que la pidieron. Cuando cada lista es un pedido, coincide con la cantidad de pedidos en que apareció, pero cuando hay *batching* una lista junta varios pedidos y la SKU se extrae una sola vez para todos, de modo que el pick cuenta visitas y no pedidos. Los *picks* dicen entonces cuántas veces hubo que ir hasta esa SKU, aunque no cuánto se caminó en cada viaje, porque eso depende de dónde está guardada, del recorrido y de qué otras SKU se buscan en la misma pasada. La segunda medida cuenta unidades, y con el tamaño de cada una, volumen y peso. Sirve para dimensionar la manipulación, la reposición y el transporte, pero no el espacio que la SKU ocupa, que depende de cuánto stock se guarda de ella y no de cuánto se mueve. En un depósito real con decenas de miles de SKU las dos medidas casi no van juntas (Bartholdi and Hackman, 2014, sección 2.3), un producto puede pedirse mucho y mover poco volumen o al revés, así que cuál mirar depende de qué se quiera decidir, los *picks* para acortar recorridos y las unidades para reposición y transporte.

Si se ordenan las SKU de la más pedida a la menos pedida, la curva que aparece suele ser similar a la de la Figura 2.5a, unas pocas SKU concentran la mayor parte de los *picks* y una cola muy larga se reparte el resto, incluso en algunos casos la quinta parte de las SKU genera más de tres cuartos de los *picks*. Esa forma es la que está detrás de la regla del 80-20, que dice que una quinta parte de las SKU suele explicar cuatro quintas partes de la actividad, y de la clasificación ABC, que corta el ranking en tres clases, las A, que son las pocas SKU que concentran la actividad, las B, una franja intermedia, y las C, el grueso del catálogo que casi no se mueve (Bartholdi and Hackman, 2014, sección 14.2). La cola larga importa aunque se mueva poco, porque por su cantidad ocupa la mayor parte del espacio, y de ahí la idea de que cada depósito es en realidad dos, uno chico y predecible donde está el trabajo y uno enorme que se lleva el espacio (Bartholdi and Hackman, 2014, sección 2.3). Qué tan marcada es la curva depende del negocio, y por ejemplo en productos de oficina o ferretería suele ser bastante plana mientras que en moda o música puede ser extrema (Bartholdi and Hackman, 2014, sección 14.2).

La otra parte del perfil es la forma de los pedidos, y ahí no alcanza con el promedio de líneas por pedido, porque el promedio esconde la mezcla y la mezcla es lo que decide la operación. Un depósito donde casi todos los pedidos tienen una sola línea y unos pocos tienen veinte puede tener el mismo promedio que otro donde todos tienen tres, pero se organizan distinto, en el primero conviene separar los pedidos de una línea, que se recolectan de a muchos por pasada sin nada que clasificar, y atender aparte a los grandes, que son pocos pero concentran buena parte de los *picks*, mientras que en el segundo todos los pedidos se tratan igual. Por eso hace falta la distribución completa, cuántos

pedidos tienen una sola línea, cuántos dos, y así, junto con la fracción de los *picks* que aporta cada tamaño. La Figura 2.5b muestra un caso típico con dos barras por cada tamaño de pedido, la azul es la fracción de los pedidos que tienen ese tamaño y la naranja la fracción de los *picks* que esos pedidos generan, y se ve que los de una línea son la mayoría de los pedidos pero una parte mucho menor del trabajo, porque cada uno aporta una sola extracción mientras que uno de cinco líneas aporta cinco. Esa distribución decide cómo organizar el picking, ya que con pocas líneas por pedido conviene agrupar varios en una misma pasada y con muchas conviene repartirlos por zonas (Bartholdi and Hackman, 2014, sección 14.2).




(a) Curva de Pareto de los *picks*

(b) Distribución de líneas por pedido

Figura 2.5: Dos vistas esquemáticas del perfil de actividad. En (a) una fracción chica de las SKU concentra la mayor parte de los *picks*, y en (b) los pedidos de una línea son la mayoría pero aportan una parte menor del trabajo.

El propio perfil también cambia con el tiempo, muchos productos tienen alguna estacionalidad, anual, mensual o semanal, y no siempre es la obvia (Bartholdi and Hackman, 2014, sección 14.2), y a eso se suman las SKU nuevas, que aparecen poco pedidas solo porque llevan poco tiempo en el catálogo (Bartholdi and Hackman, 2014, sección 14.1). Hay además una asimetría que surge de la propia forma de la curva, porque el tramo de las SKU más pedidas suele ser estable, una que fue muy pedida el mes pasado tiende a seguir siéndolo el siguiente, mientras que en la cola es imposible saber si una SKU se va a pedir, y con cantidades tan chicas cualquier fluctuación parece una tendencia (Bartholdi and Hackman, 2014, sección 2.3).

Todo lo anterior mide cada SKU de forma individual, pero los pedidos con varias líneas tienen otra información, qué cosas se piden juntas. El paso más simple trabaja con familias de productos, que son los grupos en que el propio negocio clasifica sus SKU, por proveedor, por categoría o por cómo se exhiben en la tienda, y cuenta para cada par de familias en cuántos pedidos aparecen las dos. Con eso ya se ve, por ejemplo, que casi todo pedido que trae algo de una familia trae también algo de otra, lo que sugiere guardarlas cerca (Bartholdi and Hackman, 2014, capítulo 14). Ese conteo esconde una trampa, porque dos familias pueden coincidir pocas veces en total y sin embargo una de ellas casi nunca pedirse sin la otra. Con pares el trabajo es manejable, porque con $n$ familias hay $n(n-1)/2$ pares que contar, pero con ternas la cantidad de combinaciones crece como $n^3$ y con grupos mayores explota, así que deja de ser posible contarlas

todas, y por eso la mirada de los pares complementa a la de cada SKU por separado en vez de reemplazarla.

## 2.5. Asignación de ubicaciones

La geometría dice qué ubicaciones valen más y el perfil dice qué SKU se piden más, y la asignación de ubicaciones, o slotting, es juntar las dos cosas, decidir qué SKU va en cada ubicación. A diferencia del edificio y del equipo, es una decisión que se toma sobre un depósito ya armado y que se puede volver a tomar cuando la demanda cambia, lo que la hace la más barata de corregir de todas las decisiones vistas hasta acá. Esta sección recorre las reglas con que se la resuelve, de la más simple a la que usa el historial de *picks*.

### 2.5.1. Dedicado o compartido

Hay dos niveles de decisión. El primero es si cada SKU tiene un lugar fijo, a lo que se llama almacenamiento dedicado, o si puede guardarse en cualquier ubicación libre, lo que se conoce como almacenamiento compartido. El compartido aprovecha bien el espacio, porque una ubicación que se vacía vuelve a usarse enseguida, pero dispersa cada producto por el depósito y su ubicación cambia todo el tiempo, así que exige un WMS que sepa dónde está cada cosa y disciplina para no sacarla de otro lado. El dedicado, en cambio, deja que los operarios aprendan el layout y que lo más pedido se ponga cerca, a cambio de que sus ubicaciones queden parcialmente vacías durante buena parte del ciclo, porque se llenan al reponer y se van vaciando hasta la siguiente reposición. En el caso más sencillo e idealizado, con consumo constante y reposición al vaciarse, una ubicación que recibe $Q$ unidades por ciclo de largo $T$ tiene un stock $I(t) = Q\,(1 - t/T)$, cuyo promedio es

$$\frac{1}{T}\int_0^T Q\left(1 - \frac{t}{T}\right)\,dt = \frac{Q}{2}, \tag{2.1}$$

es decir que la mitad del espacio reservado está ocioso en promedio, como muestra la Figura 2.6, y esa fracción cambia si la demanda es irregular, si se repone antes de vaciar o si se guarda stock de seguridad.

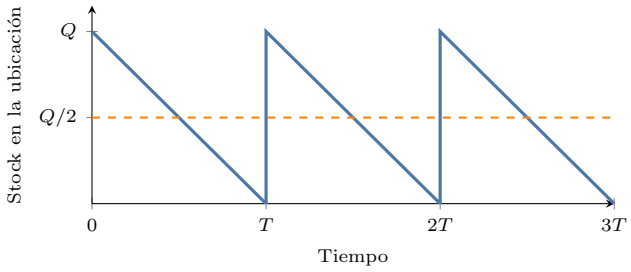



Figura 2.6: Stock de una ubicación dedicada bajo consumo constante y reposición al vaciarse. Se llena con $Q$ unidades en cada reposición y baja en línea recta hasta cero, de modo que en promedio ocupa la mitad del espacio reservado. Adaptado de Bartholdi and Hackman (2014, capítulo 2).

### 2.5.2. Las reglas de asignación

El segundo nivel es con qué regla se elige la ubicación, y las más usadas son la aleatoria, que toma cualquier ubicación libre con igual probabilidad y reparte el tráfico por todo el depósito, la de la ubicación libre más cercana al *dock*, que es lo que termina pasando cuando se deja elegir al operario y que con el tiempo llena el frente y deja el fondo vacío, y las que ordenan las SKU por su actividad, por clases o una por una, que son las que usan el historial de *picks*. De estas últimas, la asignación una por una pide almacenamiento dedicado, mientras que la de clases queda en el medio, porque reserva una zona por clase y dentro de ella deja compartir, y para elegir con cualquiera de las dos hay que empezar por medir dos cosas, qué tan lejos queda cada ubicación del punto donde empiezan y terminan los recorridos, y cuántas veces se la va a visitar, porque el costo de una asignación sale de combinar una con la otra.

Para lo primero alcanza con la distancia, porque si se supone que cada recorrido sale del *dock*, llega a una sola ubicación y vuelve, a cada ubicación $k$ le corresponde un costo de acceso $c_k$, que es la distancia caminada en ese viaje de ida y vuelta, medida sobre los pasillos desde el *dock* hasta la ubicación y de regreso, y que depende solo de la geometría, como muestran las dos ubicaciones de la Figura 2.7. Para lo segundo sirve el perfil de actividad, porque si la SKU $i$ aparece en $f_i$ listas de recolección y está guardada en la ubicación $k(i)$, la caminata que genera es proporcional a $f_i\, c_{k(i)}$ y la de todo el depósito a

$$\sum_i f_i\, c_{k(i)}, \tag{2.2}$$

donde los $c_k$ vienen dados por el edificio, y pueden medirse de distintas maneras, en línea recta con la norma euclídea, sumando el tramo a lo largo del pasillo y el transversal con la norma uno, o como el largo del camino real entre estanterías, que es la que corresponde en un depósito porque las estanterías no se atraviesan, mientras que la asignación $k(i)$ es lo que se decide (Bartholdi and Hackman, 2014, sección 6.2). Si cada viaje va del *dock* a una sola ubicación y vuelve, el costo de cada ubicación no depende de lo que haya en las demás, y por eso la suma es fácil de minimizar. Basta ordenar las ubicaciones de menor a mayor $c_k$, ordenar las SKU de más a menos $f_i$ y asignarlas en ese orden, y ningún otro reparto da una suma menor, porque si dos SKU quedaran en el orden contrario intercambiarlas nunca la aumenta. Con recorridos que visitan varias ubicaciones por viaje la suma deja de ser la caminata real y pasa a ser una aproximación, porque el costo de ir a una ubicación depende de cuáles otras se visitan en el mismo recorrido.

### 2.5.3. Picks, reposiciones y rotación

Las $f_i$ cuentan solo uno de los dos motivos por los que se visita una ubicación, por un lado la visitan los operarios que recolectan, una vez por cada lista que pide esa SKU, y por el otro la visitan los que reponen, una vez por cada vez que hay que volver a llenarla, y las dos frecuencias pueden ser muy distintas. Conviene entonces separar los *picks* $f_i$ de las reposiciones $r_i$, y la rotación del inventario, las unidades despachadas sobre las que se tienen en stock, es una tercera cosa, que va con $r_i$ y no con $f_i$. Además cuando una SKU ocupa varias ubicaciones hace falta una heurística o forma de elegir desde cuál recolectar y

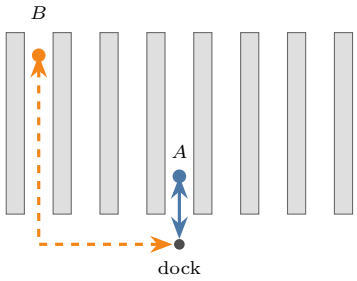



Figura 2.7: Dos ubicaciones y su viaje de ida y vuelta desde el *dock*, cada uno dibujado como una sola línea con punta en los dos extremos, con $c_A < c_B$.

en cuál reponer, la más cercana para caminar menos, la que tiene menos stock para liberarla antes, o la que tiene más para ir vaciándolas de forma pareja. Cuando se mueven *pallets* enteros y cada *pallet* que sale se repone con otro, las tres crecen a la par, porque cada *pallet* que entra se guarda una vez y se saca una vez, y de ese caso viene la regla de guardar cerca lo que más rota. Cuando se recolectan piezas, en cambio, lo que manda es $f_i$, y ordenar las SKU por *picks* no da el mismo orden que ordenarlas por rotación. Una medida que respeta esa diferencia es el *cube-per-order index* (COI), propuesto por Heskett (1963). Para cada SKU $i$ se calcula como

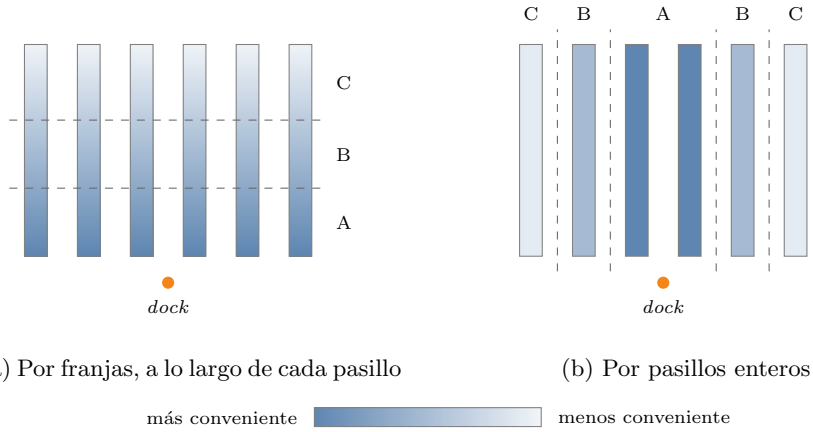
$$\mathrm{COI}_i = \frac{v_i}{f_i}, \tag{2.3}$$

donde $v_i$ es el espacio que la SKU necesita en el depósito, medido por ejemplo en metros cúbicos de estantería, y $f_i$ la cantidad de *picks* que genera en el período, así que un COI chico quiere decir muchos *picks* por cada metro ocupado. La regla es ordenar las SKU de menor a mayor COI y darles en ese orden las ubicaciones más convenientes, con lo que las mejores posiciones van a las SKU que más se piden en relación con el espacio que ocupan, y no simplemente a las que más se piden.

### 2.5.4. Clases y restricciones

Asignar una ubicación a cada SKU según su puesto en el ranking es lo más preciso, pero rara vez se hace así. Lo habitual es cortar el ranking en dos o tres clases, las A, B y C del perfil, reservar una zona del depósito para cada clase y, dentro de la zona, guardar donde haya lugar. Se pierde algo de distancia respecto de asignar una por una, pero a cambio una SKU solo se mueve cuando cambia de clase y no cada vez que cambia de puesto. Las zonas pueden dibujarse de dos maneras, que la Figura 2.8 compara sobre el mismo depósito, con las A ocupando el tramo de cada pasillo más cercano al corredor frontal, o con las A ocupando enteros los pasillos más cercanos al *dock*, y cuál conviene depende de la política de ruteo con que después se recorra, porque con la política de retorno lo que cuenta es cuánto hay que entrar en cada pasillo, y el tono de la figura sigue el costo de acceso $c_k$ de cada ubicación, mientras que con la política en $S$ lo que cuenta es en cuántos pasillos se entra, y el tono sigue el costo de entrar a cada pasillo. A todo esto se le suman restricciones que el ranking no ve, como que lo pesado vaya abajo y lo liviano arriba, que lo más pedido quede a la altura de las manos, que lo que necesita frío o es peligroso tenga su zona, que cada

ubicación tenga un tamaño, y que algunas familias que vienen dadas de afuera, por proveedor o por categoría, se prefiera mantenerlas juntas. La lista cambia de un depósito a otro, porque las restricciones vienen de cosas distintas, del producto, su peso, su tamaño y si necesita frío o es peligroso, del edificio, qué carga aguanta cada estantería y hasta qué altura llega un operario sin escalera, y de la operación, qué familias conviene tener juntas o separadas. En la práctica son ellas las que recortan, para cada SKU, el conjunto de ubicaciones donde puede ir, y el ranking decide recién dentro de ese conjunto, así que un método de asignación que no las contemple produce asignaciones que después no se pueden aplicar.




(a) Por franjas, a lo largo de cada pasillo

(b) Por pasillos enteros

Figura 2.8: Dos maneras de cortar en clases las ubicaciones de un mismo depósito, con el *dock* abajo. El tono indica la conveniencia, más oscuro cuanto más corto es el viaje de ida y vuelta desde el *dock*, y las líneas punteadas son los cortes entre las clases A, B y C. En (a) la conveniencia crece hacia el frente de cada pasillo y las clases son franjas. En (b) lo que cuenta es el pasillo, los centrales son más convenientes que los de los costados, y las clases son pasillos enteros.

### 2.5.5. Lo que queda fuera de la cuenta

Ninguna de estas reglas se puede juzgar sola, la misma asignación rinde distinto según la política de ruteo con que se recorra el depósito y cuántos pedidos se junten en cada pasada, ya que asignar SKU por SKU se nota más con pedidos chicos, mientras que con la política en $S$ y listas largas las asignaciones se parecen cada vez más entre sí, de modo que una política de almacenamiento se compara siempre con un ruteo y un batching fijos. A eso se suma la pregunta de cuándo se decide, una cosa es armar la asignación desde cero, cuando se abre un depósito o se cambia el layout, y otra es mantenerla, porque el ranking que la justificó cambia con el tiempo, entran SKU nuevas, otras dejan de pedirse y la estacionalidad mueve a las del medio, así que una asignación que fue buena se va desarmando sola. Como mover una SKU cuesta un viaje de operario, un hueco que hay que tener libre y una actualización en el WMS, en la práctica no se reasigna todo de golpe, sino que se hace por partes, o bien se aprovecha

cada reposición para guardar la SKU directamente en su lugar nuevo, y así el depósito se va corrigiendo de a poco. Por último, poner lo más pedido cerca del *dock* tiene un costo escondido, porque concentra a los operarios en los mismos pasillos a la misma hora, y en un pasillo angosto uno espera al otro, de modo que las esperas se acumulan sin que la suma de distancias las vea, y por eso una asignación algo más repartida puede funcionar mejor en la práctica que la que da la cuenta.

## 2.6. Afinidad entre productos

La demanda de un producto no es independiente de la de los demás, y esta sección se ocupa de cómo medir esa relación entre dos SKU y de qué se hace con la medida. Antes de medir nada conviene distinguir cuatro términos que se suelen usar de forma indistinta, pero que guardan ciertas diferencias.

Co-ocurrencia es un conteo, en cuántos pedidos aparecieron juntas dos SKU cualesquiera.

Asociación es dar un paso más y afirmar que cuando aparece una tiende a aparecer la otra, lo que se escribe como una regla de $i$ hacia $j$ y se evalúa con dos números, qué tan seguido se da el par y, de las veces que se pidió $i$, en cuántas también se pidió $j$.

Correlación es una medida estadística de cuánto se aleja lo que se pidió junto de lo que se esperaría si las dos demandas fueran independientes, que para dos variables de sí o no es el coeficiente $\phi$ y va de $-1$ a 1, y que puede ser baja aunque dos SKU muy pedidas coincidan mucho.

Afinidad es el término que se usa en la literatura de *slotting* para el peso que dice cuánto conviene guardar dos SKU cerca, y no es un dato sino una decisión, porque se construye con alguna de las tres anteriores.

Lo segundo que hay que fijar es qué quiere decir que dos SKU van juntas, y la opción más común es el pedido del cliente, dos SKU van juntas si alguien las compró en la misma orden. Pero el operario no camina un pedido, camina una lista de recolección, y cuando hay batching esa lista junta varios pedidos, y una *wave* junta varias listas que salen en el mismo camión. Por eso comprados juntos y recolectados juntos son dos cosas distintas, dos SKU que ningún cliente pidió a la vez podrían aparecer de forma recurrente en la misma lista, aunque sea poco probable, y dos que casi siempre se compran juntas podrían quedar repartidas en listas separadas. Si lo que se quiere bajar es la caminata, lo que corresponde es contar sobre la lista, que es lo que el operario recorre, y usar el pedido solo cuando cada lista es un pedido o cuando se comprueba que las dos cuentas dan parecido, algo que depende de cómo el algoritmo de batching arma las listas.

Con eso fijado, todas las medidas se arman con los mismos cuatro números, la cantidad $N$ de transacciones, sean pedidos o listas según la unidad elegida, la frecuencia $s_i$ y $s_j$ de cada SKU, es decir en cuántas de esas transacciones aparece cada una, y la co-ocurrencia $n_{ij}$, en cuántas aparecen las dos. Las seis medidas más usadas son las siguientes.

Co-ocurrencia, $n_{ij}$. Cuenta en cuántas transacciones aparecen las dos SKU juntas, y favorece a los pares populares, porque dos SKU muy pedidas coinciden mucho aunque no tengan nada que ver entre sí.

Soporte del par, $n_{ij}/N$. Es la co-ocurrencia como fracción de las transacciones, así que mide lo mismo en una escala de 0 a 1.

Confianza de $i$ a $j$, $n_{ij}/s_i$. Dice qué fracción de las transacciones de $i$ trae también a $j$, que es la probabilidad condicional $P(j \mid i)$ estimada del historial, y es la única de las seis que es asimétrica, porque puede ser alta de $i$ a $j$ y baja al revés cuando una es mucho más pedida que la otra.

Lift, $N\,n_{ij}/(s_i\,s_j)$. Compara las coincidencias observadas con las que se esperarían si las dos SKU fueran independientes, que salen de multiplicar la probabilidad de que una transacción traiga a $i$, $s_i/N$, por la de que traiga a $j$, y de volver a multiplicar por las $N$ transacciones, lo que da $s_i s_j/N$. Vale 1 cuando lo observado es lo esperado, más de 1 cuando van juntas más de lo que daría el azar y menos de 1 cuando se evitan, así que mide lo mismo que la correlación, el alejamiento de la independencia, aunque en otra escala y sin cota superior. Con frecuencias chicas se dispara, porque dos SKU pedidas tres veces cada una entre 10.000 transacciones tienen menos de una milésima de coincidencia esperada, y una sola coincidencia real ya da un lift de más de mil, que es justamente el régimen de la cola larga (Li et al., 2015).

Jaccard, $n_{ij}/(s_i + s_j - n_{ij})$. Mira, de todas las transacciones que trajeron a alguna de las dos, qué fracción trajo a las dos, de modo que sube a los pares raros que casi siempre van juntos y baja a los populares que coinciden mucho pero en una fracción chica de sus transacciones. En la literatura de slotting es la elección más frecuente, por convención más que por derivación (Zhang, 2016).

Coseno, $n_{ij}/\sqrt{s_i s_j}$. Hace algo parecido a Jaccard, dividiendo la co-ocurrencia por la media geométrica de las dos frecuencias, y resulta menos severo que Jaccard cuando $s_i$ y $s_j$ difieren mucho.

Que estas medidas no necesariamente responden a la misma pregunta puede verse con dos pares. Supongamos 10.000 pedidos, dos SKU populares $A$ y $B$, con frecuencias de 2.000 y 1.500 que coinciden en 300 pedidos, y dos SKU no tan populares $C$ y $D$, con frecuencias de 12 y 10 que coinciden en 9. La Tabla 2.3 da las seis medidas para cada par.

| **Par** | **Co-ocurr.** | **Soporte** | **Confianza** | **Lift** | **Jaccard** | **Coseno** |
|---|---|---|---|---|---|---|
| $A$ y $B$ (populares) | 300 | 0,03 | 0,15 | 1 | 0,09 | 0,17 |
| $C$ y $D$ (raras) | 9 | 0,0009 | 0,75 | 750 | 0,69 | 0,82 |

Tabla 2.3: Las seis medidas para dos pares de SKU sobre 10.000 pedidos, uno popular que coincide 300 veces y uno raro que coincide 9. La confianza es la de la primera SKU hacia la segunda.

Las dos medidas absolutas favorecen al par popular, mientras que todas las normalizadas dan vuelta el orden, y el caso más claro es el lift, que para el par popular vale exactamente 1, porque sus 300 coincidencias son justo las que se

esperarían por azar, y para el par raro vale 750. Si esos valores se usan como peso $a_{ij}$ para decidir qué acercar, con co-ocurrencia el par popular pesa 300 contra 9 y el modelo prioriza acercar a $A$ y $B$, mientras que con cualquiera de las otras medidas prioriza a $C$ y $D$, y en ninguno de los dos casos acercarlos garantiza un ahorro en cada una de esas transacciones, porque la caminata real depende de toda la lista. No son entonces dos versiones de un mismo objetivo sino dos objetivos distintos, uno cuenta viajes conjuntos y el otro qué tan exclusiva es la relación, y elegir el peso es elegir qué se está minimizando.

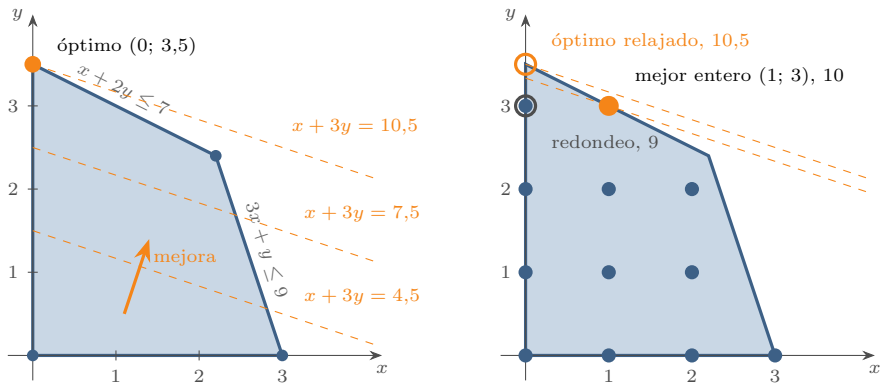
Todo lo anterior mira de a dos SKU por vez, y esa es la simplificación más común de la literatura, porque la cantidad de pares ya crece de forma cuadrática con la cantidad de SKU y la de ternas o grupos mayores explota, aunque existen algoritmos que encuentran los conjuntos de productos que aparecen juntos con frecuencia sin tener que enumerarlos todos (Han et al., 2004). La afinidad tampoco tendría por qué salir del historial de pedidos, en entornos de manufactura se la ha derivado de la estructura del producto, de qué piezas forman parte de un mismo conjunto, con una señal mucho más nítida que la del comercio electrónico (Brynzér and Johansson, 1996), e incluso podría estimarse con modelos de aprendizaje automático a partir de los atributos de los productos.

Con cualquiera de las medidas simétricas, que son todas las anteriores salvo la confianza, el resultado es una matriz simétrica de pesos $a_{ij}$, que en un depósito con muchas SKU y pedidos chicos suele ser además rala, porque la mayoría de los pares nunca coincide. Las dos cosas terminan otorgando una ventaja, mientras que la simetría hace que cada par aporte un solo término al costo, con lo que alcanza con sumar sobre $i < j$, y lo ralo hace que ese costo tenga pocos términos distintos de cero, de modo que mover una SKU solo cambia los términos de sus vecinos. En la práctica no se guarda la matriz entera, porque la mayoría de sus entradas distintas de cero son ruido, pares que coincidieron una o dos veces por casualidad y que las medidas normalizadas, el lift en particular, inflan. Lo habitual es podarla de una de dos maneras, con un umbral, descartando los pares cuya co-ocurrencia o soporte no llega a un mínimo, o por vecindad, conservando para cada SKU solo sus $k$ vecinas de mayor afinidad, y las dos dejan una matriz rala, con del orden de $k\,n$ entradas en lugar de $n^2$, que es lo que vuelve barato evaluar el costo y recalcularlo cuando se mueve una SKU. El precio es elegir el corte, porque un umbral alto o un $k$ chico descartan pares reales de la cola larga, y uno demasiado laxo deja pasar el ruido que se quería sacar.

## 2.7. Formulación matemática

Las herramientas para escribir y resolver estas decisiones vienen de la optimización lineal. Un programa lineal tiene un conjunto de variables reales, una función objetivo que es una combinación lineal de esas variables y que se quiere minimizar, y un conjunto de restricciones lineales, igualdades o desigualdades, que las variables tienen que cumplir, y cualquier problema de ese tipo puede llevarse a una forma estándar, minimizar el objetivo sujeto a un sistema de igualdades lineales y a que ninguna variable sea negativa. Cada restricción recorta un semiespacio, así que la región factible, el conjunto de puntos que cumplen todas, es la intersección de finitos semiespacios, es decir un poliedro, y esa geometría es la que vuelve tratable al problema, porque como el objetivo es lineal su mínimo se alcanza en el borde de la región, y si la región es acota-

da y no vacía el mínimo existe y hay al menos un vértice del poliedro que lo alcanza, como en la Figura 2.9a. Los vértices son finitos, así que en principio alcanzaría con evaluarlos todos, pero su cantidad crece de forma combinatoria con la cantidad de restricciones, y lo que hace el método símplex, que Dantzig (1963) propuso en 1947, es recorrerlos con criterio, arrancando en un vértice y pasando a un vértice vecino que mejore el objetivo, hasta llegar a uno sin vecinos mejores, que por la convexidad del problema es el óptimo global. En el peor caso el símplex puede visitar una cantidad exponencial de vértices, pero en la práctica resuelve problemas con miles de variables en segundos, y desde los años ochenta se conocen además algoritmos de punto interior que garantizan tiempo polinomial (Karmarkar, 1984). Cuando las variables tienen que ser enteras, como cuando deciden si una SKU va o no a una ubicación, el problema se vuelve en general mucho más difícil, porque los vértices del poliedro relajado, el que se obtiene al olvidar la condición de integralidad, ya no tienen por qué ser enteros, y como muestra la Figura 2.9b el mejor punto entero puede no ser un vértice ni el redondeo del óptimo relajado, así que hay que buscar entre las soluciones enteras con métodos de ramificación, salvo en algunas familias afortunadas en las que la relajación alcanza.




(a) El programa lineal

(b) Con variables enteras

Figura 2.9: Un programa lineal en dos variables y su versión entera. En (a) la región factible es el polígono que recortan las restricciones, las líneas punteadas son curvas de nivel del objetivo $x+3y$, que en este ejemplo se maximiza, la flecha indica hacia dónde mejora y el óptimo se alcanza en un vértice. En (b) solo valen los puntos de coordenadas enteras, marcados en azul. El vértice óptimo de la relajación no es entero, el mejor punto entero no es un vértice del polígono, y tampoco es el redondeo del óptimo relajado, que vale menos.

La suma de la Ecuación 2.2 se puede escribir como un problema de asignación. Con $I$ el conjunto de SKU y $L$ el de ubicaciones, la variable $x_{i\ell}$ vale 1 si

la SKU $i$ va a la ubicación $\ell$ y 0 si no, y el problema es

$$\begin{aligned} \min_{x} \quad & \sum_{i \in I} \sum_{\ell \in L} f_i \, c_\ell \, x_{i\ell} \\ \text{sujeto a} \quad & \sum_{\ell \in L} x_{i\ell} = 1 \quad \forall i \in I \\ & \sum_{i \in I} x_{i\ell} \leq 1 \quad \forall \ell \in L \\ & x_{i\ell} \in \{0, 1\}, \end{aligned} \tag{2.4}$$

donde la primera restricción dice que cada SKU ocupa exactamente una ubicación, que es el caso del almacenamiento dedicado con un solo lugar por SKU, y si alguna necesita varias se la trata como varias copias, la segunda que cada ubicación aloja a lo sumo una SKU, con lo que puede haber ubicaciones vacías, y la tercera que la decisión es sí o no. El costo de poner a $i$ en $\ell$ es $f_i \, c_\ell$ y no depende de dónde estén las demás, así que el objetivo es lineal en las $x_{i\ell}$, y el problema es uno de los clásicos de la investigación de operaciones, el problema de asignación. Es además una de esas familias afortunadas, porque si se relaja la última restricción y se deja que las variables tomen cualquier valor entre 0 y 1, la solución óptima del programa lineal sigue siendo entera, así que se resuelve de forma exacta con el símplex o con algoritmos propios como el húngaro (Kuhn, 1955), y en este caso ni siquiera hace falta, porque el orden por $f_i$ y por $c_\ell$ de la Sección 2.5 da directamente la solución óptima.

La afinidad rompe esa independencia, porque si conviene que $i$ y $j$ estén cerca, lo que cuesta poner a $i$ en $\ell$ depende de dónde haya quedado $j$. Con $d_{\ell k}$ la distancia entre las ubicaciones $\ell$ y $k$, cada par $(i, j)$ paga su afinidad multiplicada por la distancia entre las ubicaciones que recibieron, y sumando sobre todos los pares queda

$$\sum_{i \in I} \sum_{j \in I} \sum_{\ell \in L} \sum_{k \in L} a_{ij} \, d_{\ell k} \, x_{i\ell} \, x_{jk}, \tag{2.5}$$

donde cada término lleva el producto de dos variables, $x_{i\ell}$ y $x_{jk}$, y por eso se lo llama cuadrático. Como $a_{ij} = a_{ji}$ y $d_{\ell k} = d_{k\ell}$, cada par aparece dos veces en esa suma, una como $(i, j)$ y otra como $(j, i)$, lo que en un problema puramente cuadrático es solo un factor dos que no cambia el óptimo. Ahí está todo el cambio, ya no alcanza con ordenar dos listas, porque mover una SKU cambia el costo de todas las que tienen afinidad con ella, y una mejora local puede empeorar el total.

Juntar los dos términos da un problema de la forma

$$\begin{aligned} \min_{x} \quad & \sum_{i \in I} \sum_{\ell \in L} f_i \, c_\ell \, x_{i\ell} + \sum_{i \in I} \sum_{j \in I} \sum_{\ell \in L} \sum_{k \in L} a_{ij} \, d_{\ell k} \, x_{i\ell} \, x_{jk} \\ \text{sujeto a} \quad & \sum_{\ell \in L} x_{i\ell} = 1 \quad \forall i \in I \\ & \sum_{i \in I} x_{i\ell} \leq 1 \quad \forall \ell \in L \\ & x_{i\ell} \in \{0, 1\}, \end{aligned} \tag{2.6}$$

que es la forma que Koopmans y Beckmann le dieron en 1957 al problema de ubicar plantas industriales, con un costo de instalar cada planta en cada sitio,

un flujo de mercadería entre cada par de plantas y una distancia entre cada par de sitios, y que se conoce como problema de asignación cuadrática (Koopmans and Beckmann, 1957). En el slotting los papeles son los mismos, las SKU son las plantas, las ubicaciones son los sitios, la afinidad $a_{ij}$ hace de flujo y el término lineal es el de la Ecuación 2.4. Escribirlo así, sin embargo, deja dos cosas abiertas. Una es el conteo de los pares, porque aquel factor dos que no cambiaba el óptimo del término cuadrático solo, acá duplica el peso de la afinidad frente al término lineal si no se lo corrige. La otra es la escala, porque $f_i$ cuenta *picks* y puede valer miles mientras que $a_{ij}$ depende de la medida elegida, por ejemplo Jaccard varía entre 0 y 1, así que sumar los dos términos tal cual deja que el más grande domine la optimización, por lo que lo habitual es llevar $f$ y $a$ a fracciones, de este modo cada suma pasa a ser una distancia promedio, y agregar un peso que diga cuánto importa una frente a la otra. Las dos dependen de qué se elija como $a_{ij}$ y de cómo se cuenten los pares, así que no las fija la forma general sino cada formulación concreta.

El precio de esa generalidad es la dificultad, porque el problema de asignación cuadrática es NP-hard (Sahni and Gonzalez, 1976), lo que quiere decir que es al menos tan difícil como cualquier problema de la clase NP, la de los problemas cuya solución se puede verificar en tiempo polinomial, así que un algoritmo que lo resolviera en tiempo polinomial daría uno para todos los problemas de esa clase, algo que se considera muy improbable, y hasta hoy no se conoce ninguno. En la práctica los métodos exactos que existen crecen muy rápido con el tamaño de la instancia, y como a diferencia de la asignación lineal la relajación no da soluciones enteras, no hay atajo. Para tener una idea de la escala, treinta SKU en treinta ubicaciones ya son más de $10^{32}$ asignaciones posibles, mientras que un depósito real tiene decenas de miles de cada una.

El método exacto de referencia es *branch and bound* (Land and Doig, 1960), que en lugar de probar las asignaciones una por una las organiza en un árbol, donde cada rama fija una parte de la asignación, calcula para cada rama una cota de lo mejor que podría dar y descarta las ramas cuya cota ya es peor que la mejor solución conocida. Eso recorta mucho el árbol pero no cambia su crecimiento, y la revisión de Loiola et al. (2007) ubicaba el límite práctico en el orden de las pocas decenas de objetos.

Como en instancias grandes no se conoce el óptimo, la literatura mide la calidad de una solución contra dos referencias. Una es QAPLIB, una biblioteca de instancias del problema con la mejor solución conocida para cada una, que sirve para comparar métodos entre sí. La otra es una cota inferior, un número que ninguna asignación puede bajar, y la más usada es la de Gilmore y Lawler (Gilmore, 1962; Lawler, 1963), que sale de resolver una serie de problemas de asignación lineal, aunque se vuelve floja a medida que la instancia crece y las dos referencias sirven sobre todo en instancias chicas.

Por eso, en la escala de un depósito, la solución exacta queda descartada y lo que se usa son heurísticas, que buscan soluciones buenas sin garantía de que sean las mejores, y la revisión de Loiola et al. (2007) las ordena en unas pocas familias. Conviene fijar tres ideas antes de nombrarlas. Una búsqueda local parte de una asignación cualquiera y prueba cambios chicos, como intercambiar dos SKU de lugar, quedándose con cada cambio que baja el costo hasta que ninguno lo baja, y la asignación en la que se detiene es un óptimo local, mejor que todas sus vecinas pero no necesariamente la mejor de todas. Una heurística constructiva arma la asignación de a una SKU, eligiendo en cada paso la ubicación más barata dado

lo que ya se asignó, y es rápida pero lo que entrega es solo un punto de partida, que ni siquiera tiene por qué ser un óptimo local. Una metaheurística es una estrategia que le agrega a la búsqueda local algún mecanismo para salir de un óptimo local y seguir buscando, y las más usadas en el slotting con afinidad son el recocido simulado, que acepta a veces un cambio que empeora, la búsqueda tabú, que recuerda por dónde ya pasó para no volver, y los algoritmos genéticos, que trabajan con varias soluciones a la vez y las combinan, aunque cada una trae parámetros que hay que calibrar (Li et al., 2015). La descomposición, por último, parte el problema en dos más chicos, y en el slotting toma la forma del esquema de dos fases que domina la literatura, agrupar primero las SKU afines y asignar después los grupos, como esquematiza la Figura 2.10, a costa de que lo que se pierde al separar las dos decisiones rara vez se mide (Islam and Uddin, 2023). Casi todas terminan con una búsqueda local por intercambios, que es barata porque la diferencia en la Ecuación 2.6 al permutar dos SKU se calcula sin recomputar toda la suma, y que en la práctica es lo que pule cualquier solución inicial.

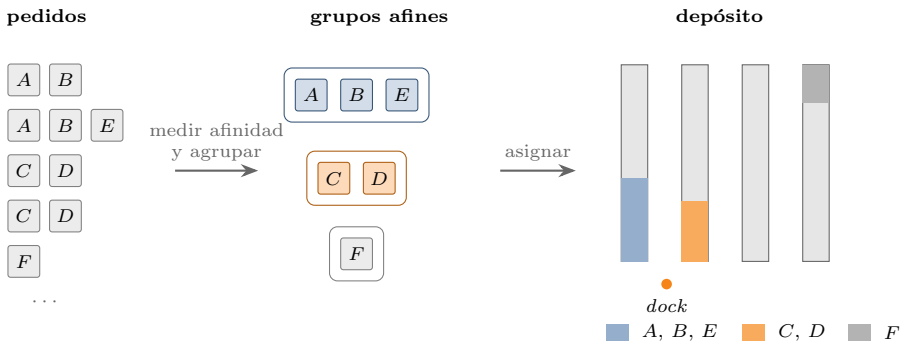



Figura 2.10: El esquema de dos fases que domina la literatura. Del historial de pedidos se mide qué SKU aparecen juntas y con eso se arman grupos de SKU afines. Recién después se decide dónde va cada grupo en el depósito, los grupos con más actividad cerca del *dock* y las SKU sin pareja donde quede lugar. Las dos decisiones se toman por separado, y lo que se pierde al separarlas rara vez se mide.

## 2.8. Estado del arte

Guardar juntos los productos que se piden juntos es una idea vieja, y como antecedente directo se suele citar a Frazelle and Sharp (1989), que a fines de los ochenta propusieron la asignación correlacionada. Desde entonces el problema recibió varios nombres, asignación correlacionada, slotting por afinidad o *family grouping*, en un área cuya terminología las propias revisiones señalan como no unificada (Rojas Reyes et al., 2019; Islam and Uddin, 2023). La revisión de Islam and Uddin (2023) cubre sesenta trabajos entre 1985 y 2022, y además del esquema de dos fases de la Sección 2.7 marca que muy pocos validan sobre datos reales. Lo que sigue no recorre esos trabajos uno por uno, sino que los ordena por cuatro preguntas que cada uno tuvo que responder, de dónde sale

la señal de afinidad, con qué método se pasa de ella a una asignación, bajo qué condiciones se prueba y con qué datos y contra qué baseline se compara, y cierra con la mejora que cada uno reporta, leída con esas condiciones, que es lo que la Tabla 2.4 resume columna por columna.

Señal de afinidad. La respuesta más común es alguna de las medidas de la Sección 2.6, sin que ninguna se haya impuesto por un argumento, ya que Zhang (2016) recoge seis medidas distintas de correlación y Li et al. (2015) combinan el lift con el conteo de soporte. Entre los trabajos recientes aparecen el conteo de co-ocurrencia por par (Islam and Uddin, 2024; Debold, 2025), las reglas de asociación junto con la clasificación ABC (Benaglia et al., 2024), y la línea de scattered storage, que reparte copias de cada SKU por el depósito y no define un peso por par, porque la proximidad entre las SKU de una misma orden queda implícita en las órdenes con que se evalúa (Gámez Albán et al., 2024; Chen et al., 2026). Ninguno de los trabajos revisados verifica antes si la co-ocurrencia observada excede la que daría el azar dados los soportes, así que la afinidad se calcula y se usa sin preguntarse si hay algo que explotar.

Método de asignación. La norma es la descomposición en dos fases de la Figura 2.10. La crítica más directa viene de Mirzaei et al. (2021), que en un sistema parts-to-picker alternan entre agrupar y asignar y reportan que la versión integrada supera a la secuencial en un 13 % promedio, un número que no se traslada al depósito manual pero que muestra que separar las dos decisiones tiene un costo. Quienes atacan la formulación cuadrática completa lo hacen con heurísticas, como Mantel et al. (2007), cuyo heurístico queda en promedio 2,14 % por encima del óptimo en 550 instancias chicas, con algoritmos genéticos (Li et al., 2015; Guan and Li, 2018), o con las variantes recientes, una que agrupa las SKU por pasillo y después ordena grupos y pasillos (Islam and Uddin, 2024), una de tres etapas con búsqueda de vecindad variable (Debold, 2025) y una búsqueda que decide a la vez dónde va cada SKU y cómo se recorre el depósito (Chen et al., 2026). Lo que separa a unos de otros no es la familia de método sino cuánto del problema resuelven de una vez y cuánto descomponen.

Condiciones de prueba. La primera condición es el tipo de sistema, porque la mayoría trabaja en picker-to-parts mientras que Mirzaei et al. (2021) y Guan and Li (2018) lo hacen en parts-to-picker, donde lo que cuesta no es caminar sino cuántos contenedores hay que traer hasta el operario. El layout va de un solo bloque de pasillos (Zhang, 2016) a varios bloques (Islam and Uddin, 2024) y al *zone picking* secuencial, donde cada operario recolecta la parte del pedido que cae en su zona y lo pasa a la zona siguiente (Debold, 2025), la política en $S$ domina el ruteo por su simplicidad (Islam and Uddin, 2023), y solo Chen et al. (2026) prueban una misma asignación bajo varias políticas. El batching casi no aparece, Mantel et al. (2007) lo nombran solo para justificar que se recolecte de a una orden por vez, con el argumento de que un batching extenso apaga el efecto de cualquier slotting, pero no lo prueban, y de los trabajos de la tabla solo Benaglia et al. (2024) lo incluyen en su simulación. El tamaño de orden va de una a veinte líneas, listas chicas incluso en el extremo alto.

Datos y baseline. Los *baselines*, es decir las asignaciones de referencia contra las que se mide la mejora, son casi siempre los tres de la Sección 2.5, aleatorio, rotación o COI, y clases ABC. Los datos son casi siempre sintéticos con la correlación plantada, ya sea generando después de cada producto de una orden otro correlacionado con una fuerza que se barre entre el 2 % y el 10 % (Xu and Ren, 2020), o manipulando la probabilidad de pedir un producto correlacionado en cinco niveles (Mirzaei et al., 2021), y las excepciones con datos reales son pocas, como el archivo de órdenes con el que trabaja Debold (2025). La escala acompaña, porque las instancias van de 12 SKU (Mantel et al., 2007) a entre 300 y 500 (Mirzaei et al., 2021) y a 1.000 (Zhang, 2016), lejos de las decenas de miles de un depósito real.

Mejora reportada. La mejora de 2,08 % que Zhang (2016) reporta sobre *full-turnover*, la asignación por rotación pura, viene de órdenes generadas solo por popularidad, sin correlación plantada, así que toda la co-ocurrencia que su método explota se explica por las frecuencias de cada SKU y ese 2 % es lo que se obtuvo en esas condiciones, y en el mismo trabajo las clases ABC quedan 10,28 % peor que full-turnover porque dentro de cada clase guardan al azar. En Xu and Ren (2020) el ahorro cae de 10,48 % con órdenes chicas a 5,0 % con órdenes de veinte líneas, porque con listas largas hay que pasar por casi todos los pasillos y la asignación ya no reduce cuántos se atraviesan, y en Mantel et al. (2007) la ventaja de asignar con cuidado frente al aleatorio se reduce a un tercio al pasar de órdenes de dos a órdenes de diez. Mirzaei et al. (2021), en parts-to-picker, quedan 6,5 % por debajo de la clasificación ABC cuando la curva de demanda es 20/80, y sus ahorros crecen con el tamaño de orden, al revés que en Xu and Ren (2020). Eso no es una contradicción sino dos regímenes, porque cuando los productos viajan hasta el operario en estanterías móviles una orden más grande podría dar más oportunidades de que las SKU pedidas compartan una misma estantería, mientras que en picker-to-parts obliga a pasar por más pasillos. Por eso la mejora porcentual por sí sola no compara trabajos si cambian el sistema, el ruteo, el tamaño de orden, los datos o el baseline, que es lo que la Tabla 2.4 deja ver.

## 2.9. Síntesis y problemas abiertos

Leída en conjunto, la evidencia anterior muestra que el aporte de la afinidad depende fuertemente de las condiciones en que se la usa, y permite señalar cuatro, cada una con su propio mecanismo. Rinde cuando los pedidos son chicos y se recolectan de a uno o en lotes chicos, porque una lista larga pasa por casi todos los pasillos de todos modos y acercar dos SKU deja de ahorrar pasillos, que es lo que Xu and Ren (2020) miden al agrandar la orden en picker-to-parts con ruteo en $S$, y lo que Mantel et al. (2007) asumen, sin probarlo, cuando dicen que un batching extenso apaga cualquier slotting. Hace falta además que la política de ruteo deje lugar al ahorro, porque con una política en $S$ estricta lo que cuenta es en cuántos pasillos se entra y no dónde está cada SKU dentro del pasillo, y cabe esperar, aunque ningún trabajo revisado lo mida, que con retorno o con recorridos que se adaptan a la lista la posición fina vuelva a importar. Tampoco alcanza si la curva de demanda está tan sesgada que ordenar por *picks* ya resuelve casi todo, porque si unas pocas SKU concentran la actividad,

| **Trabajo** | **Sistema** | **Señal** | **Método** | **Datos y baseline** | **Mejora reportada** |
|---|---|---|---|---|---|
| Mantel et al. (2007) | n/d, módulo vertical de bandejas, 12 SKU | n/d | heurístico QAP, una orden por vez | sintéticos, 550 instancias; óptimo y aleatorio | gap al óptimo 2,14 %; la ventaja sobre el aleatorio cae a un tercio con órdenes de 10 |
| Li et al. (2015) | n/d | lift y soporte | QAP con afinidad y ABC, genético | caso de estudio | 7,14 % a 104,48 % en tiempo de picking |
| Zhang (2016) | picker-to-parts, un bloque, 1.000 SKU | correlación por pares, medida n/d | estrategia correlacionada | sintéticos sin correlación plantada; full-turnover y ABC | 2,08 % menos distancia; ABC 10,28 % peor que full-turnover |
| Xu and Ren (2020) | picker-to-parts, ruteo en $S$, órdenes de hasta 20 | n/d | asignación dinámica entre *waves* | sintéticos, correlación plantada de 2 % a 10 % | 10,48 % con órdenes chicas, 5,0 % con órdenes de 20 |
| Mirzaei et al. (2021) | parts-to-picker, 300 a 500 SKU | score de afinidad | agrupar y asignar integrado | sintéticos, cinco niveles; dos fases y ABC | 13 % sobre dos fases; 6,5 % peor que ABC con curva 20/80 |
| Islam and Uddin (2024) | picker-to-parts, varios bloques | conteo de co-ocurrencia por par | agrupa por pasillo y ordena grupos y pasillos | sintéticos, 100.000 órdenes; full-turnover, ABC y la estrategia de Zhang | 22,55 % a 39,67 % menos distancia que full-turnover |
| Debold (2025) | picker-to-parts, zone picking | co-ocurrencia | tres etapas con VNS | reales; benchmarks | hasta 19,5 % |
| Benaglia et al. (2024) | depósito frío, con batching y ruteo | reglas de asociación y ABC | simulación de 8 estrategias | simulados de órdenes reales; aleatorio | hasta 8 % menos tiempo de picking |
| Gámez Albán et al. (2024) | picker-to-parts, scattered storage | implícita en las órdenes | VNS | sintéticos, 68 instancias con clases ABC; scatter aleatorio y por volumen | hasta 36 % y 56 % menos distancia |
| Chen et al. (2026) | picker-to-parts, scattered, tres ruteos | implícita en las órdenes | MIP y búsqueda adaptativa con ruteo | n/d; práctica estándar | hasta 14 % menos en el objetivo, distancia ponderada de guardado y picking |

Tabla 2.4: Condiciones y resultados de los trabajos revisados. Las celdas n/d son las que la fuente no informa o no pudieron verificarse, y la última columna solo se lee junto con las anteriores.

guardarlas cerca del *dock* captura la mayor parte del ahorro posible y la afinidad solo puede repartir lo que queda. Eso es lo que Mirzaei et al. (2021) encuentran con una curva 20/80, aunque en un sistema parts-to-picker. Y rinde poco si la co-ocurrencia observada no es más que un reflejo de la popularidad, porque dos SKU muy pedidas coinciden mucho por azar, y una afinidad construida sobre eso apenas reordena lo que la rotación ya ordenó, que es el 2 % que Zhang (2016) obtiene con órdenes generadas sin correlación. Ninguna de esas cuatro condiciones la documentó un solo trabajo, cada una aparece en uno distinto, en sistemas y con ruteos que no siempre coinciden y no siempre se informan, y ninguno de los trabajos revisados las barrió juntas, así que el umbral a partir del cual conviene pasar de la rotación a la afinidad sigue sin estar medido.

A esas cuatro se les suman otras variables que pesan tanto o más sobre lo que la afinidad puede dar, y que los trabajos de la Tabla 2.4 casi no miran. Una es la congestión, que la Sección 2.5 ya presentó como costo escondido de la rotación y que con la afinidad podría agravarse, aunque esto es un razonamiento y no un resultado, porque juntar las SKU que se piden juntas también juntaría a los operarios que las buscan. Por eso una asignación con menos distancia teórica podría resultar peor en la práctica si concentra el tráfico en unos pocos pasillos, y ninguno de los trabajos revisados mide esperas. Otra es el costo de mover la mercadería hasta la asignación nueva, el mismo que la Sección 2.5 describió al hablar de mantener una asignación, y que de los trabajos revisados solo Xu and Ren (2020) incluyen en la cuenta, al reubicar entre *waves* y cargar al tiempo del operario el sacar y volver a colocar cada producto. A eso se agrega la reposición, que sigue ocurriendo después de asignar y que con SKU agrupadas por afinidad habría que adaptar, algo que Mirzaei et al. (2021) señalan para su sistema parts-to-picker sin cuantificarlo y dejan como trabajo futuro junto con la asignación dinámica. Y queda la estabilidad en el tiempo, porque la asignación se calcula con un historial y se usa hacia adelante, lo que supone que los pares que hoy van juntos van a seguir yendo juntos meses después, un supuesto que ningún trabajo revisado mide, aunque la Sección 2.4 ya mostró que la demanda de cada SKU por sí sola cambia con la estacionalidad y con las altas. A todo esto hay que agregarle que el efecto del tamaño de orden cambia de signo entre los dos trabajos que lo miden, uno en picker-to-parts y otro en parts-to-picker, como la Sección 2.8 ya mostró, así que cualquiera de estas condiciones se lee dentro de un régimen y no se traslada al otro.

Que esas variables falten no es casualidad de cada trabajo sino un patrón de cómo se estudia el problema. Casi todo se mira de a dos SKU, la simplificación que la Sección 2.6 ya señaló, y aunque Islam and Uddin (2023) lo marcan como hueco abierto, casi ninguno de los trabajos revisados usa conjuntos de tres o más productos para asignar. Los datos son casi siempre sintéticos con la correlación plantada, como mostró la Sección 2.8, de modo que la pregunta de si hay señal que explotar viene respondida por construcción, y la evaluación se hace casi siempre sobre un slotting estático, muchas veces en un solo bloque de pasillos, sin esperas, sin reposición y con una afinidad que no cambia en el tiempo. Cada una de esas decisiones es razonable por separado, porque aísla el efecto que se quiere medir, pero todas parecen ir en la misma dirección, hacia una señal más nítida y un depósito más quieto que uno en operación, y por eso las mejoras reportadas se leen mejor como lo que la afinidad podría dar bajo esas condiciones que como lo que da en operación. Y, como la Sección 2.8 ya marcó, ninguno de los trabajos revisados contrasta la co-ocurrencia observada contra la que darían

SKU pedidas de forma independiente con los soportes observados, que es lo que separaría una relación real de una coincidencia explicada por las frecuencias, como la de Zhang (2016).

De ahí salen los problemas que siguen abiertos, y el primero es la escala, porque las instancias de la Tabla 2.4 van de la docena al millar de SKU, con órdenes de hasta veinte líneas, mientras que un depósito de comercio electrónico puede tener decenas de miles de SKU y, cuando hay batching, listas de recolección que juntan varios pedidos, y no está claro que los métodos ni las conclusiones sobrevivan al salto. También falta validar la señal antes de usarla, comparando la co-ocurrencia observada con la que se obtendría si cada lista se armara al azar conservando la frecuencia de cada SKU y el tamaño de cada lista, y rompiendo solo la relación entre SKU, una comparación que ninguno de los trabajos revisados hace y que decide si hay algo que optimizar. Tampoco está resuelta la afinidad dinámica, tanto en el sentido de cuánto dura una afinidad medida como en el de cómo reasignar cuando cambia, porque entre los trabajos revisados casi no aparece, ya que Xu and Ren (2020) reasignan entre *waves* con datos sintéticos y Mirzaei et al. (2021) lo dejan como trabajo futuro. Otro costo sin medir es el de descomponer en dos fases, que Mirzaei et al. (2021) cuantificaron en parts-to-picker pero que en el depósito manual, donde la descomposición es la norma, ninguno de los trabajos revisados mide. Y queda la elección del peso $a_{ij}$, que la Sección 2.6 mostró que no es una cuestión de escala sino de qué objetivo se minimiza, y que en los trabajos revisados casi siempre se toma por convención y rara vez se discute qué pares termina acercando.

Este trabajo se ubica en esos huecos, ya que trabaja sobre un depósito manual picker-to-parts de comercio electrónico real, con decenas de miles de SKU y listas de recolección que juntan varios pedidos, es decir en la escala y en el régimen que los trabajos de la Tabla 2.4 no cubren. Mide la señal antes de usarla, para saber si la co-ocurrencia de sus listas excede la de ese modelo nulo, y compara la afinidad contra la rotación bajo las condiciones del propio depósito, con su ruteo, su batching y su curva de demanda, y no bajo condiciones armadas para aislar el efecto. Se pregunta, por último, si una mejora en la suma de la Ecuación 2.6 se traduce en distancia recorrida, que es la diferencia entre el objetivo y lo que se quiere bajar que la Sección 2.5 dejó planteada. Para eso hace falta primero conocer el depósito y los datos con que se hizo todo lo anterior, que es lo que sigue.

# Bibliografía


Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, Princeton.

Ballou, R. H. (2007). The evolution and future of logistics and supply chain management. *European Business Review*, 19(4):332–348.

Bartholdi, J. J. and Hackman, S. T. (2014). *Warehouse & Distribution Science*. Supply Chain and Logistics Institute, Georgia Institute of Technology. Release 0.96.

Benaglia, M. F., Chen, M.-H., Lu, S.-H., Tsai, K.-M., and Hung, S.-H. (2024). Improving the picking efficiency of a cold warehouse to avoid temperature abuse. *The International Journal of Logistics Management*, 35(5):1434–1464.

Boysen, N., de Koster, R., and Weidinger, F. (2019). Warehousing in the e-commerce era: A survey. *European Journal of Operational Research*, 277(2):396–411.

Brown, S. A. (1997). *Revolution at the Checkout Counter: The Explosion of the Bar Code*. Harvard University Press.

Brynzér, H. and Johansson, M. I. (1996). Storage location assignment: Using the product structure to reduce order picking times. *International Journal of Production Economics*, 46-47:595–603.

Chen, R., Yang, J., and Yu, Y. (2026). Integrated scattered storage and picker routing in picker-to-parts warehouses. *European Journal of Operational Research*, 329(3):808–824.

Dantzig, G. B. (1963). *Linear Programming and Extensions*. Princeton University Press, Princeton.

de Koster, R., Le-Duc, T., and Roodbergen, K. J. (2007). Design and control of warehouse order picking: A literature review. *European Journal of Operational Research*, 182(2):481–501.

Debold, S. (2025). A storage assignment approach considering SKU repetitions in sequential zone picking. *Computers & Industrial Engineering*, 210:111482.

Frazelle, E. H. and Sharp, G. P. (1989). Correlated assignment strategy can improve any order-picking operation. *Industrial Engineering*, 21(4):33–37.

Gámez Albán, H. M., Cornelissens, T., and Sörensen, K. (2024). A new policy for scattered storage assignment to minimize picking travel distances. *European Journal of Operational Research*, 315(3):1006–1020.

Gilmore, P. C. (1962). Optimal and suboptimal algorithms for the quadratic assignment problem. *Journal of the Society for Industrial and Applied Mathematics*, 10(2):305–313.

Gu, J., Goetschalckx, M., and McGinnis, L. F. (2007). Research on warehouse operation: A comprehensive review. *European Journal of Operational Research*, 177(1):1–21.

Guan, M. and Li, Z. (2018). Genetic algorithm for scattered storage assignment in Kiva mobile fulfillment system. *American Journal of Operations Research*, 8:474–485.

Gue, K. R. and Meller, R. D. (2009). Aisle configurations for unit-load warehouses. *IIE Transactions*, 41(3):171–182.

Han, J., Pei, J., Yin, Y., and Mao, R. (2004). Mining frequent patterns without candidate generation: A frequent-pattern tree approach. *Data Mining and Knowledge Discovery*, 8:53–87.

Heskett, J. L. (1963). Cube-per-order index: A key to warehouse stock location. *Transportation and Distribution Management*, 3(4):27–31.

Islam, M. S. and Uddin, M. K. (2024). An efficient correlation-based storage location assignment heuristic for multi-block multi-aisle warehouses. *International Journal of Industrial Engineering and Management*, 15(2):125–139.

Islam, S. M. and Uddin, K. M. (2023). Correlated storage assignment approach in warehouses: A systematic literature review. *Journal of Industrial Engineering and Management*, 16(2):294–318.

Karmarkar, N. (1984). A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–395.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York.

Koopmans, T. C. and Beckmann, M. (1957). Assignment problems and the location of economic activities. *Econometrica*, 25(1):53–76.

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97.

Land, A. H. and Doig, A. G. (1960). An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520.

Lawler, E. L. (1963). The quadratic assignment problem. *Management Science*, 9(4):586–599.

Li, J., Moghaddam, M., and Nof, S. Y. (2015). Dynamic storage assignment with product affinity and ABC classification: a case study. *International Journal of Advanced Manufacturing Technology*, 84:2179–2194.

Loiola, E. M., de Abreu, N. M. M., Boaventura-Netto, P. O., Hahn, P., and Querido, T. (2007). A survey for the quadratic assignment problem. *European Journal of Operational Research*, 176(2):657–690.

Mantel, R. J., Schuur, P. C., and Heragu, S. S. (2007). Order oriented slotting: a new assignment strategy for warehouses. *European Journal of Industrial Engineering*, 1(3):301–316.

Mirzaei, M., Zaerpour, N., and de Koster, R. (2021). The impact of integrated cluster-based storage allocation on parts-to-picker warehouse performance. *Transportation Research Part E: Logistics and Transportation Review*, 146:102207.

Ratliff, H. D. and Rosenthal, A. S. (1983). Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem. *Operations Research*, 31(3):507–521.

Rojas Reyes, J. J., Solano-Charris, E. L., and Montoya-Torres, J. R. (2019). The storage location assignment problem: A literature review. *International Journal of Industrial Engineering Computations*, 10:199–224.

Sahni, S. and Gonzalez, T. (1976). P-complete approximation problems. *Journal of the ACM*, 23(3):555–565.

Tompkins, J. A., White, J. A., Bozer, Y. A., and Tanchoco, J. M. A. (2010). *Facilities Planning*. John Wiley & Sons, 4 edition.

Womack, J. P., Jones, D. T., and Roos, D. (1990). *The Machine That Changed the World*. Rawson Associates, New York.

Xu, X. and Ren, C. (2020). Research on dynamic storage location assignment of picker-to-parts picking systems under traversing routing method. *Complexity*, 2020:1621828.

Zhang, Y. (2016). Correlated storage assignment strategy to reduce travel distance in order picking. *IFAC-PapersOnLine*, 49(2):30–35.

# Índice de figuras

2.1. La unidad de manipulación se reduce a medida que el producto avanza por la cadena de distribución. . . . . 8
2.2. Planta esquemática de un depósito y el camino de un producto. Entra por el *dock* de recepción (1), el *put-away* lo lleva hasta una ubicación de las estanterías (2), donde queda almacenado (3) hasta que un recorrido de *picking*, que sale de empaque (punto azul), pasa a buscarlo (4) y lo deja en empaque (5), de donde sale por el *dock* de despacho (6). En naranja el tramo de entrada, en azul el de salida. . . . . 10
2.3. Variantes de geometría de un depósito. La ubicación de los *docks* y la orientación de los pasillos cambian qué ubicaciones resultan convenientes, y en (a) y (b) el sombreado más oscuro indica mayor conveniencia. Esquemas adaptados de Bartholdi and Hackman (2014, capítulo 6). . . . . 13
2.4. Dos políticas de ruteo sobre el mismo layout, con las estanterías en gris y las mismas cuatro extracciones (puntos). La política en $S$ recorre completos solo los pasillos con extracciones, mientras que la de retorno entra y sale de cada uno por el corredor frontal. La línea punteada es la vuelta al inicio. . . . . 15
2.5. Dos vistas esquemáticas del perfil de actividad. En (a) una fracción chica de las SKU concentra la mayor parte de los *picks*, y en (b) los pedidos de una línea son la mayoría pero aportan una parte menor del trabajo. . . . . 17
2.6. Stock de una ubicación dedicada bajo consumo constante y reposición al vaciarse. Se llena con $Q$ unidades en cada reposición y baja en línea recta hasta cero, de modo que en promedio ocupa la mitad del espacio reservado. Adaptado de Bartholdi and Hackman (2014, capítulo 2). . . . . 18
2.7. Dos ubicaciones y su viaje de ida y vuelta desde el *dock*, cada uno dibujado como una sola línea con punta en los dos extremos, con $c_A < c_B$. . . . . 20

2.8. Dos maneras de cortar en clases las ubicaciones de un mismo depósito, con el *dock* abajo. El tono indica la conveniencia, más oscuro cuanto más corto es el viaje de ida y vuelta desde el *dock*, y las líneas punteadas son los cortes entre las clases A, B y C. En (a) la conveniencia crece hacia el frente de cada pasillo y las clases son franjas. En (b) lo que cuenta es el pasillo, los centrales son más convenientes que los de los costados, y las clases son pasillos enteros. . . . . 21
2.9. Un programa lineal en dos variables y su versión entera. En (a) la región factible es el polígono que recortan las restricciones, las líneas punteadas son curvas de nivel del objetivo $x + 3y$, que en este ejemplo se maximiza, la flecha indica hacia dónde mejora y el óptimo se alcanza en un vértice. En (b) solo valen los puntos de coordenadas enteras, marcados en azul. El vértice óptimo de la relajación no es entero, el mejor punto entero no es un vértice del polígono, y tampoco es el redondeo del óptimo relajado, que vale menos. . . . . 25
2.10. El esquema de dos fases que domina la literatura. Del historial de pedidos se mide qué SKU aparecen juntas y con eso se arman grupos de SKU afines. Recién después se decide dónde va cada grupo en el depósito, los grupos con más actividad cerca del *dock* y las SKU sin pareja donde quede lugar. Las dos decisiones se toman por separado, y lo que se pierde al separarlas rara vez se mide. . . . . 28

# Índice de tablas

2.1. Tipos de centro de distribución según el cliente al que sirven, y lo que cada uno implica para el pedido, el surtido y la unidad que se despacha. Elaborado a partir de Bartholdi and Hackman (2014, sección 1.2) y, para las devoluciones del comercio electrónico, de Boysen et al. (2019). . . . . 7
2.2. Reparto típico del costo operativo entre etapas, según las estimaciones clásicas de Tompkins et al. (2010) reproducidas en Bartholdi and Hackman (2014, capítulo 3). . . . . 11
2.3. Las seis medidas para dos pares de SKU sobre 10.000 pedidos, uno popular que coincide 300 veces y uno raro que coincide 9. La confianza es la de la primera SKU hacia la segunda. . . . . 23
2.4. Condiciones y resultados de los trabajos revisados. Las celdas n/d son las que la fuente no informa o no pudieron verificarse, y la última columna solo se lee junto con las anteriores. . . . . 31

# Glosario

COI *cube-per-order index*, espacio que necesita una SKU dividido por las veces que se la pide . . . . . 20

SKU *stock keeping unit*, referencia de producto que el inventario administra por separado . . . . . 6

WMS *warehouse management system*, sistema de gestión del depósito . . . . . 8